

ROYAL NAVY · SITE EDUCATIVO



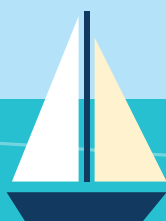
MODO CLARO



MODO ESCURO

Documentação Técnica Completa

Auditoria, desempenho, identidade visual e modo escuro
— dossiê de construção do site, versão 2.0



Repositório

github.com/AlanSouzaDev7/Naval_Design-WEB-V.1

Versão

Versão 2.0 · 26 de setembro de 2026 · tag v2.0.0 · GitHub Pages

Sumário

1	Resumo executivo	resultados, métricas e o que foi entregue (v1.1 e v2.0)
2	Dossiê técnico: como o site foi construído	arquitetura, arquivos e o papel de cada linguagem
3	O motor 3D em detalhe	Three.js, WebGL, ciclo de vida das cenas e temas
4	O hero animado e as demais animações	Canvas 2D, controlador de cenas, reveal, contadores, modal
5	Modo escuro: guerra em alto mar (v2.0)	botão, transição, batalha, mergulho, naufrágios e desempenho
6	Diagnóstico de desempenho	o que deixava o site lento e como foi resolvido
7	Erros e bugs corrigidos	catálogo completo com causa e solução
8	Paletas: dia ensolarado e noite de guerra	cores, aplicação e acessibilidade
9	Como foi verificado	metodologia, medições e limitações
10	Pendências e recomendações	o que ficou de fora e próximos passos
11	Operação: executar, versionar e publicar	comandos, variáveis, versões e GitHub
12	Publicação, domínio e segurança	GitHub Pages, royalnavy.com.br, CSP e checklist
13	Glossário	termos técnicos usados neste documento

Como ler este documento. As seções 1 a 4 explicam *o que é o site e como ele funciona*; a seção 5 é o dossiê do **modo escuro** (novidade da versão 2.0); as seções 6 e 7 registram *o que estava errado e o que foi feito*; a seção 8 documenta as duas identidades visuais; as seções 9 a 12 cobrem verificação, pendências, operação e a publicação na internet (incluindo o domínio próprio). Todos os números citados foram medidos ou extraídos do código-fonte durante o trabalho — nada é estimativa sem indicação.

O que mudou nesta edição (v2.0). O documento da versão 1.1 foi mantido e atualizado; foram acrescentados o dossiê completo do modo escuro (seção 5), o registro dos ajustes feitos durante o desenvolvimento dele (seção 7.5), a paleta noturna (seção 8), as novas verificações e medições (seção 9), o histórico de versões (seção 11) e novos termos no glossário.

1. Resumo executivo

O projeto **Royal Navy** é um site educativo sobre a Marinha Real Britânica, com linha do tempo, navios lendários em **3D interativo**, batalhas, frota atual e galeria. É servido por Flask (Python) e escrito em HTML, CSS e JavaScript puros, com o Three.js para os modelos 3D. A partir da **versão 2.0** ele tem **dois modos**, escolhidos por um botão no canto superior direito: o **modo claro** (um dia ensolarado de navegação, igual ao da versão 1.1) e o **modo escuro** (“guerra em alto mar”: batalha naval no topo, mergulho ao descer a página, combates submarinos e navios naufragados no fim).

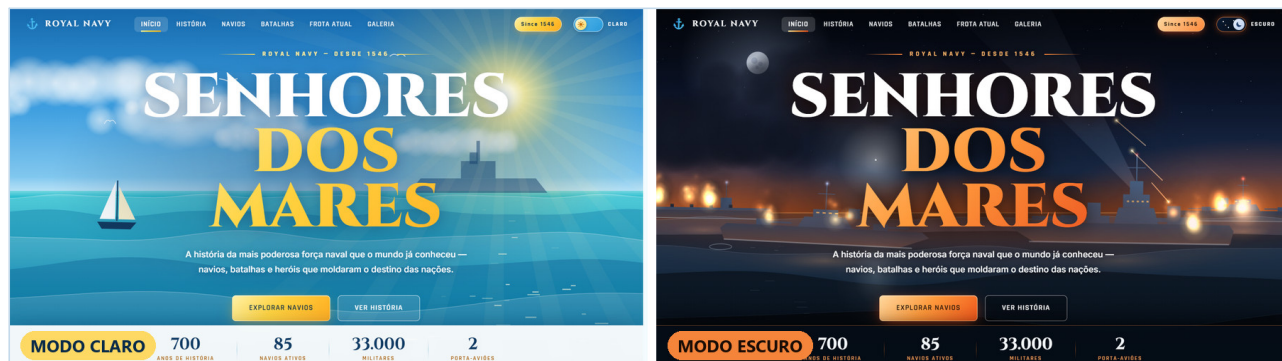


Figura 1 — O mesmo hero nos dois modos (1366×768). Esquerda: modo claro, idêntico ao da v1.1, agora com o botão de tema. Direita: modo escuro, com a batalha naval em curso.

Foram solicitadas duas grandes entregas, ambas concluídas:

- **Versão 1.x — auditoria e nova identidade:** leitura completa do código; diagnóstico de lentidão e de animações pouco fluidas (medido no navegador); correções de desempenho, bugs funcionais, servidor e acessibilidade; paleta “dia ensolarado de navegação”; pré-visualização; publicação no GitHub e **site no ar pelo GitHub Pages** (HTTPS), com preparação segura para o domínio royalnavy.com.br (seção 12).
- **Versão 2.0 — modo escuro “Guerra em alto mar”:** botão de tema com animação de transição; hero em batalha naval; mergulho até o leito do mar com medidor de profundidade; faixas de guerra submarina e de navios naufragados; cenas 3D noturnas — tudo **animado e otimizado**, sem alterar o modo claro (seção 5).

Resultados medidos — versões 1.0 → 1.1

Indicador	Antes (v1.0)	Depois (v1.1)
FPS medido no painel do navegador (DPR 1,25; WebGL por software)	1,7	56,7 no topo · 60 com os navios visíveis
Contextos WebGL criados ao carregar a página	13, todos renderizando em 60 fps continuamente	0 (criados sob demanda ao chegar perto da tela)
Draw calls do 3D com o hero na tela (nenhum navio visível)	Contínuos (todas as cenas rodavam sempre)	0
Draw calls por frame com o modal aberto	Fundo + modal	≈ 17 (só o modal; fundo pausado)
Redimensionamento de canvas 3D	A cada frame em telas com DPR ≠ 1	Somente quando o tamanho muda
Pesos de fonte baixados do Google Fonts	12	9
Hover dos cards	Não funcionava	Funciona (translateY -8 px)
Barras da seção Frota	Ficavam em 0 para sempre	Animam até o valor correto
Conteúdo do modal (Armamento, História)	Cortado, sem rolagem	Rolável (1192 px de conteúdo em 691 px)
Erros no console do navegador	—	Nenhum, em desktop e celular
Acesso ao site	Só em localhost:5000, na máquina do autor	Público, em HTTPS: alansouzadev7.github.io/Naval_Design-WEB-V.1

Sobre essa medição: o painel do navegador do aplicativo renderiza WebGL por software (sem placa de vídeo), então os valores absolutos são pessimistas — o mesmo teste, mesma máquina e mesma configuração, foi usado antes e depois, o que torna a *comparação* válida. Em uma GPU real o ganho tende a ser menor em FPS, mas continua significativo em uso de CPU/GPU e bateria, porque o trabalho contínuo deixou de existir.

Resultados medidos — versão 2.0

Medidas feitas em uma máquina modesta (Intel Core i3-1215U, gráficos integrados Intel UHD, 7,7 GB de RAM), com o Edge acelerado por GPU (D3D11), janela de 1366×768 e DPR 1. Cada valor é a **mediana de 3 rodadas**, contando quadros do *requestAnimationFrame* durante 3 s. O monitor da máquina atualiza a 120 Hz; uma página sem nenhum canvas desenhando chega a esse teto (≈ 120 fps).

Cenário (fps · ms por quadro)	v1.1 · claro	v2.0 · claro	v2.0 · escuro
Topo: hero animado (canvas 2D em tela cheia)	62 · 16,2	61 · 16,3	61 · 16,3 (batalha naval)
Grade de navios (6 cenas WebGL visíveis)	93 · 10,8	82 · 12,2	3D noturno: mesma ordem de grandeza
Seção sem nenhum canvas (Batalhas)	120 · 8,3	120 · 8,3	120 · 8,3 (só o mar profundo)
Faixa de guerra submarina	— (não existe)	— (oculta)	120 · 8,3
Faixa de navios naufragados	— (não existe)	— (oculta)	90 · 11,1

Como ler: no modo claro a v2.0 fica na mesma faixa da v1.1 (as diferenças entre versões ficam dentro da variação entre rodadas, que chega a ±10% nas cenas 3D nesta máquina). No modo escuro, o hero em batalha custa o mesmo que o hero claro; ao descer, os trechos sem 3D chegam ao teto do monitor porque o fundo do mar roda em meia resolução e as faixas só desenhavam quando aparecem na tela (seção 5.8). Detalhes, rolagem contínua e limitações na seção 9.

Outros resultados da v2.0

Indicador	Resultado
Modo claro: diferença de pixels contra a v1.1 publicada (corpo das seções História, Linha do tempo e Batalhas)	0 pixel (comparação pixel a pixel, mesmas condições) — a única diferença visível é o novo botão na barra de navegação
Modo claro: alterações no CSS original (style.css)	0 linhas alteradas ; 116 linhas <i>acrescentadas</i> (botão, transição e ocultação dos elementos do escuro)
Canvases que desenhavam no modo claro	Somente o do hero. Fundo do mar, faixas submarinas e medidor têm <code>display:none</code> : não medem layout nem executam laço
Peso adicional (5 arquivos novos)	≈ 100 KB brutos · ≈ 29 KB com gzip (o GitHub Pages comprime); scripts com <code>defer</code> , só o CSS escuro bloqueia a pintura (4 KB gzip)
Erros de console e violações de CSP	Nenhum, nos dois modos, no Flask local, na versão estática e no site publicado
Persistência e troca	Tema salvo e reaplicado antes da 1ª pintura (sem “pisar”); cliques repetidos durante a animação são ignorados sem quebrar o estado
Publicação	Commit ae6b9ea + tag v2.0.0; GitHub Pages atualizado em ≈ 15 s e verificado no ar

Tamanho do código

Arquivo	v1.0 (original)	v1.1	v2.0	Observação (v2.0)
static/css/style.css	1569	1579	1695	Só acréscimos (116 linhas): botão, transição, ocultar itens do escuro
static/css/dark.css	—	—	406	Novo: todo o visual do modo escuro
static/js/main.js	464	617	668	Controlador RNHero + cena clara registrada + ganchos de tema
static/js/ships-3d.js	1034	867	915	Temas claro/escuro para o 3D
static/js/ships-data.js	182	182	182	Sem alterações
static/js/theme-init.js	—	—	11	Novo: aplica o tema antes da 1ª pintura
static/js/theme.js	—	—	140	Novo: botão, transição, desempenho adaptativo
static/js/war-hero.js	—	—	741	Novo: batalha naval do hero
static/js/deep-sea.js	—	—	791	Novo: mergulho, guerra submarina, naufrágios, medidor
templates/index.html	545	557	609	Botão, canvases, faixas, scripts
app.py	41	70	70	Sem alterações na v2.0
Total de JavaScript	1680	1666	3448	Dobrou: o modo escuro é ≈ 1700 linhas novas

Linhas contadas com `wc -l` nos commits da v1.1 (d006295) e da v2.0 (ae6b9ea). Na v1.1 o `app.py` já estava com os cabeçalhos de segurança (70 linhas).

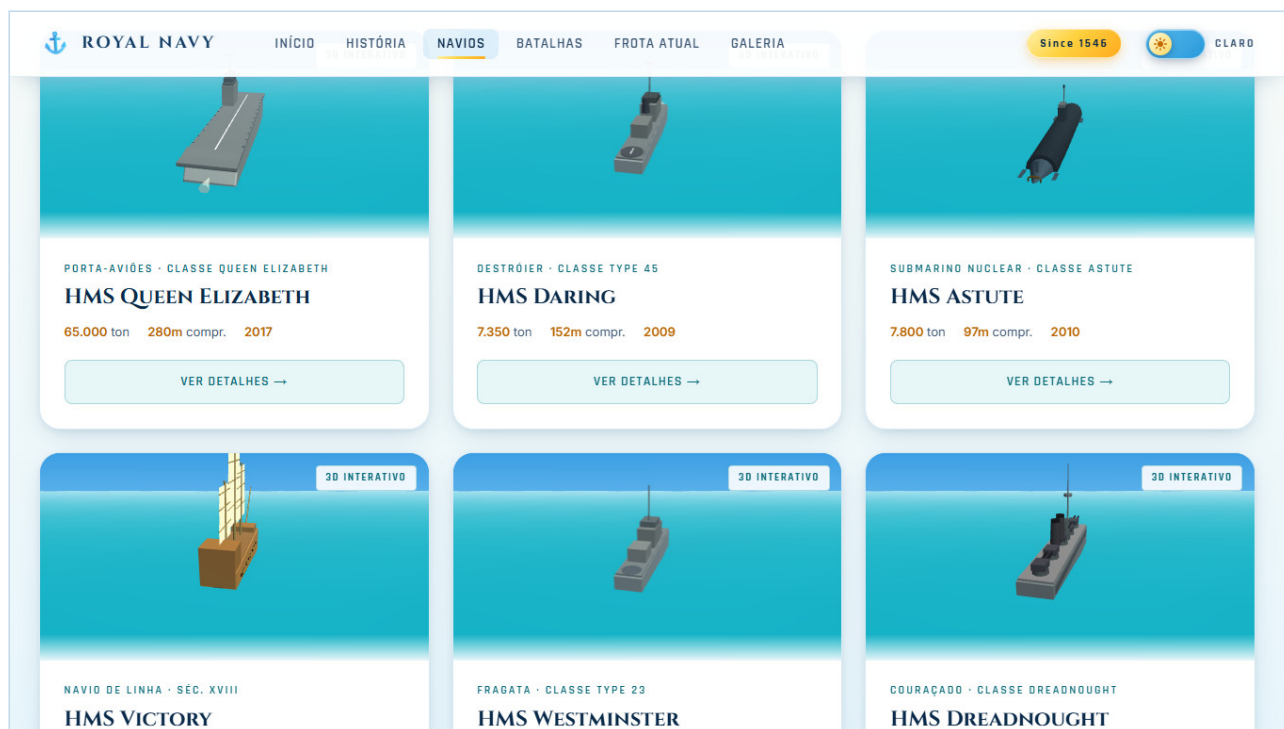
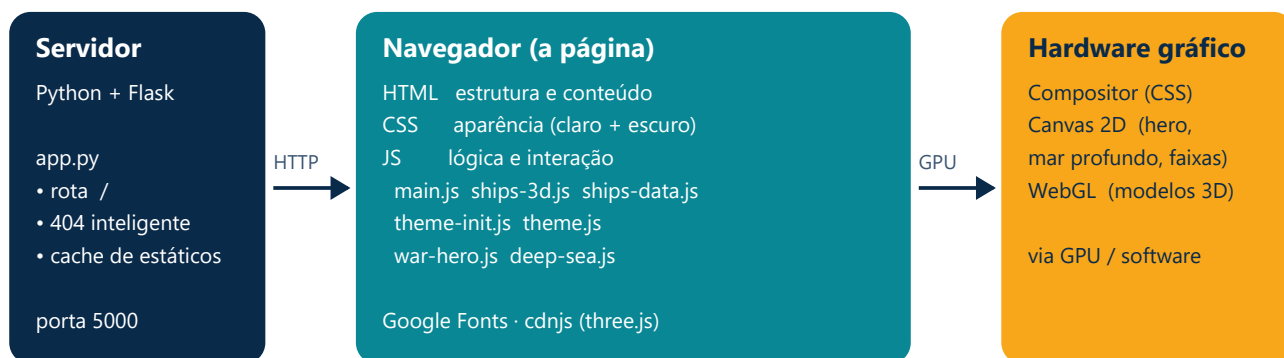


Figura 2 — Modo claro, seção Navios: cards com cenas 3D de mar ensolarado, iluminadas por sol e céu (idêntica à v1.1).

2. Dossiê técnico: como o site foi construído

2.1 Visão geral da arquitetura

O site é uma **página única** (single page) servida por um pequeno servidor Python. O servidor entrega o HTML e os arquivos estáticos; **toda a interatividade acontece no navegador**, em três camadas que trabalham juntas: estrutura (HTML), aparência (CSS) e comportamento (JavaScript). O JavaScript se divide em responsabilidades distintas — interface, hero em canvas 2D, 3D em WebGL, dados e, desde a v2.0, **tema** (botão, transição e cenas do modo escuro).



Fluxo: o navegador pede "/" → Flask devolve index.html → o navegador baixa CSS, fontes e scripts → o JavaScript monta a interação e pede à GPU para desenhar. Depois do primeiro carregamento, o servidor não participa mais. O tema escolhido fica no navegador do visitante (localStorage) e é aplicado antes da primeira pintura.

2.2 Quem faz o quê: o papel de cada tecnologia

Tecnologia	Responsabilidade no projeto	Onde vive
Python 3 + Flask	Servidor web. Entrega a página em /, os arquivos de /static, define cache, trata 404 e decide se o modo debug pode ficar ligado. Não gera conteúdo dinâmico: o template não tem lógica Jinja. Serve os arquivos do modo escuro sem nenhuma alteração.	app.py
HTML5	Estrutura semântica e todo o conteúdo em português: navegação, hero, linha do tempo, cards de navios, batalhas, frota, galeria, rodapé e o modal. Define os <canvas> onde o 3D e as cenas 2D são desenhados, ganchos de dados (data-*) e atributos de acessibilidade. Na v2.0: o botão de tema (role="switch"), o canvas do mar profundo, o medidor de profundidade e as duas faixas submarinas.	templates/index.html
CSS3	Toda a aparência: paleta (variáveis CSS), tipografia, grid/flex, cards, botões, ondas divisórias, responsividade (4 breakpoints), estados de hover/foco, transições e animações de entrada. Na v2.0 há dois arquivos : style.css (modo claro + botão + transição) e dark.css (tudo que só vale com data-theme="dark").	static/css/style.css · dark.css
JavaScript — interface	Comportamento da página: navbar que muda ao rolar, link ativo, parallax do hero, contadores, revelação dos elementos ao rolar, filtro de navios, barras da frota, modal (abrir/fechar/foco/ESC), brilho do cursor.	static/js/main.js
JavaScript — tema	Aplica o tema salvo antes da 1ª pintura, liga o botão, executa a animação circular de troca, avisa o resto do site (evento rn:themechange) e mede o desempenho para reduzir efeitos em máquinas lentas (RNPerf).	theme-init.js · theme.js
JavaScript — Canvas 2D	Desenha o hero (controlador RNHero com uma cena por tema: dia ensolarado ou batalha naval), o fundo fixo do mar profundo, a guerra submarina, os naufrágios e o medidor de profundidade, sempre com animação baseada em tempo.	main.js · war-hero.js · deep-sea.js
JavaScript + Three.js (WebGL)	Constrói e renderiza os modelos 3D: 6 tipos de navio, oceano, luz, câmera e interação de arrastar para girar. Gerencia o ciclo de vida das 12 cenas + modal e agora troca céu, névoa, luzes e água entre o cenário diurno e o noturno.	static/js/ships-3d.js
JavaScript — dados	Base de dados dos navios (nome, classe, descrição, especificações, armamento, história) que alimenta o modal.	static/js/ships-data.js
SVG (embutido)	Ícone do site (favicon), a máscara em onda usada nos divisores de seção e os ícones de sol e lua do botão de tema — como data: URI ou SVG em linha, sem requisições extras.	index.html, style.css
Serviços externos	Google Fonts (Cinzel, Inter, Rajdhani) e cdnjs (three.js r128, com verificação SRI). O modo escuro não adiciona nenhuma origem externa.	CDN

2.3 Estrutura de arquivos

```

site/
├── app.py                servidor Flask (70 linhas)
├── requirements.txt      flask==3.0.3
├── README.md            uso, modos claro/escuro, publicação e domínio
├── build_static.py      gera docs/ (versão estática)
├── enable_custom_domain.py ativa/remove domínio (trava de DNS)
├── docs/               site estático do GitHub Pages (11 arquivos)
├── documentacao/       este PDF
├── .gitignore
├── templates/
│   └── index.html      a página inteira (609 linhas)
├── static/
│   ├── css/
│   │   ├── style.css   modo claro + botão + transição (1695 linhas)
│   │   └── dark.css     modo escuro (406 linhas) [v2.0]
│   └── js/
│       ├── theme-init.js tema antes da 1ª pintura (11 linhas) [v2.0]
│       ├── theme.js     botão, transição, RNPerf (140 linhas) [v2.0]
│       ├── main.js      interface + controlador do hero (668 linhas)
│       ├── ships-3d.js   motor 3D Three.js + temas (915 linhas)
│       ├── ships-data.js dados dos navios (182 linhas)
│       ├── war-hero.js   batalha naval do hero (741 linhas) [v2.0]
│       └── deep-sea.js   mergulho, submarinos, naufrágios (791) [v2.0]

```

2.4 Seções da página e o que cada uma usa

Seção (id)	Conteúdo	Tecnologias envolvidas
Navbar	Marca, links âncora, selo “Since 1546” e botão de tema	CSS (fixa, troca de estilo ao rolar) · JS (classe <i>scrolled</i> , link ativo) · theme.js (botão)
Hero (#hero)	Título, chamada, botões, estatísticas	Canvas 2D (claro: dia de sol · escuro: batalha naval) · CSS (raios de sol, vinheta, animações de entrada) · JS (parallax, contadores)
História (#history)	Linha do tempo com 7 marcos	CSS (grid/flex, cards) · JS (reveal ao rolar) · <i>escuro</i> : fundo = canvas fixo do mar profundo
Navios (#ships)	Filtro + 6 cards com 3D	Three.js/WebGL (cenário diurno ou noturno) · CSS · JS (filtro, drag, modal)
Guerra Submarina (<i>só no escuro</i>)	Submarinos, torpedos, sonares, cargas de profundidade	Canvas 2D (#sub-canvas) · deep-sea.js
Batalhas (#battles)	6 batalhas, uma em destaque	CSS (grid, ondas em máscara SVG; no escuro as ondas somem e o fundo é o mar)
Frota (#fleet)	Cena 3D em formação + 6 categorias	WebGL · CSS (barras animadas por transform)
Galeria (#gallery)	5 cenas 3D (1 grande)	WebGL · CSS grid
Navios Naufragados (<i>só no escuro</i>)	Cascos no leito, algas, cardumes, baliza	Canvas 2D (#wreck-canvas) · deep-sea.js
Modal	Ficha técnica + 3D em tela cheia	HTML (dialog) · JS (foco, ESC) · WebGL (renderer reutilizado)
Rodapé	Links e créditos	CSS (onda divisória no claro; “leito do mar” no escuro)
Medidor de profundidade (<i>só no escuro</i>)	Metros e zona oceânica, à direita, sob a navbar	HTML + CSS (HUD fixo) · deep-sea.js (Gauge)

2.5 Ordem de carregamento (o que acontece ao abrir a página)

- **1.** O navegador pede `/`; o Flask devolve `index.html` (≈ 29 KB).
- **2.** Ele lê o `<head>`: primeiro o `theme-init.js` (síncrono, 0,5 KB), que define `data-theme` a partir do que ficou salvo — assim a página já *nasce* no tema certo, sem piscar. Depois vêm o ícone embutido, *preconnect* para Google Fonts e *cdnjs*, a folha de fontes (9 pesos) e os dois CSS (`style.css` ≈ 49 KB e `dark.css` ≈ 18 KB). O CSS é bloqueante de renderização — por isso a página já nasce estilizada.
- **3.** Os sete scripts têm *defer*: baixam em paralelo *sem bloquear* a pintura e executam em ordem — `theme.js` $\rightarrow$ `three.js` $\rightarrow$ `ships-data.js` $\rightarrow$ `ships-3d.js` $\rightarrow$ `main.js` $\rightarrow$ `war-hero.js` $\rightarrow$ `deep-sea.js`.
- **4.** `theme.js` liga o botão; `main.js` liga os módulos (`scroll`, `hero`, `contadores`, `reveal`, `filtro`, `modal...`), cria o controlador `RNHero` com a cena clara e chama `ShipEngine.init()`; `war-hero.js` registra a cena escura no controlador; `deep-sea.js` prepara o fundo do mar, as faixas e o medidor. No modo claro tudo isso fica **dormente**.
- **5.** `init()` apenas *registra* os 12 canvases 3D. Nenhuma cena é criada ainda; isso só ocorre quando um canvas chega a 120 px da tela.

2.6 Decisões de projeto

Decisão	Motivo
Manter Flask, HTML/CSS/JS puros e Three.js r128	Preservar a base do projeto; r128 é a última série com o build global <code>three.min.js</code> , que o site usa via CDN.
Paleta em variáveis CSS (<code>:root</code>)	Trocar o tema inteiro editando um bloco; os tokens têm nomes semânticos (<code>sky</code> , <code>sea</code> , <code>sun</code> , <code>navy</code> , <code>coral</code> , <code>sand</code>).
Só <code>transform</code> e <code>opacity</code> em animações	Essas duas propriedades são animadas pelo compositor da GPU, sem recalculando layout nem repintar.
Estado visual por classes , não por <code>style inline</code>	Evita brigas de especificidade (causa de dois bugs originais) e mantém o CSS como fonte única da aparência.
Um único loop de animação 3D	Um ponto de controle para pausar/retomar tudo (aba oculta, modal aberto, fora da tela).
Servidor só em <code>127.0.0.1</code> por padrão	O debugger do Werkzeug executa código no servidor; nunca deve ficar exposto na rede.
v2.0 · Tema como atributo no <code><html></code> + arquivo CSS separado (<code>dark.css</code>)	O modo claro não foi tocado (só acréscimos no <code>style.css</code>). Isso garante que o claro continue idêntico ao aprovado — e foi provado por comparação de pixels.
v2.0 · Um único canvas de hero, com cenas registradas por tema	Nada de segundo canvas nem de segundo laço de animação: trocar de tema é trocar qual cena o controlador desenha.
v2.0 · Fundo do mar em <i>um</i> canvas fixo, de meia resolução	A cor da água, os raios de luz e as partículas custam ¼ dos pixels; o conteúdo da página (cards quase opacos) fica por cima.
v2.0 · Faixas submarinas com canvas próprio, só ativas quando visíveis	As duas cenas mais pesadas (submarinos e naufrágios) não custam nada enquanto estão fora da tela ou no modo claro.
v2.0 · Sprites pré-renderizados e <i>pools</i> de partículas	Nenhuma alocação por quadro: sem pausas do coletor de lixo e sem criar gradientes a cada frame.
v2.0 · Qualidade adaptativa (RNPerf)	Em máquinas lentas o site reduz partículas e efeitos sozinho, em vez de travar; o visitante não precisa configurar nada.
v2.0 · <code>theme-init.js</code> externo e síncrono	Aplica o tema antes da 1ª pintura e é compatível com a CSP (que proíbe scripts inline).
v2.0 · View Transitions API com cortina de reserva	Melhoria progressiva: navegadores modernos ganham a revelação circular nativa; os demais, uma cortina equivalente.
v2.0 · Tema padrão sempre <i>claro</i>	O modo claro é o visual atual do site; a preferência de cores do sistema operacional não é consultada — vale a escolha salva do visitante.

3. O motor 3D em detalhe

Todo o 3D está em `ships-3d.js`, dentro de uma função anônima que expõe apenas `window.ShipEngine = { init, openModal, closeModal, setTheme }`. Se o `three.js` não carregar (CDN indisponível), o motor vira um *no-op* e o restante do site continua funcionando.

3.1 Modelos procedurais

Nenhum arquivo de modelo (`.glb`, `.obj`) é usado: cada navio é **montado por código** a partir de primitivas (caixas, cilindros, cones, esferas, planos e uma extrusão para o casco do porta-aviões).

Função	Modelo	Peças notáveis
<code>buildCarrier()</code>	Porta-aviões (Queen Elizabeth)	Casco extrudado, convés, rampa <i>ski-jump</i> , ilha com 3 níveis, radar, 26 luzes de borda
<code>buildDestroyer()</code>	Destróier (Type 45)	Casco + proa, superestrutura, ponte, canhão, VLS, helipad
<code>buildFrigate()</code>	Fragata (Type 23)	Casco, superestrutura, canhão, mastro, helipad
<code>buildSubmarine()</code>	Submarino (Astute)	Casco cilíndrico, cúpula, cone de popa, vela, periscópio, 4 tubos, lemes, hélice
<code>buildGalleon()</code>	Galeão (HMS Victory)	Casco, 3 mastros com 9 velas/vergas, 16 portinholas, 8 canhões, gurupés, bandeira
<code>buildBattleship()</code>	Couraçado (Dreadnought)	Casco, 5 torres de canhões gêmeos, 2 chaminés, mastro com cofa

3.2 Cena, luz e cenário

- **Céu:** uma textura de gradiente (2×256 px, criada em um canvas 2D) usada como `scene.background` e **compartilhada** por todas as cenas do mesmo tema (uma textura por tema, guardada em cache).
- **Mar:** plano de 80×80 com 30×30 segmentos e ondulação determinística (gerador pseudoaleatório com semente fixa); material opaco. A linha d'água fica em $y = -0,32$, de modo que os cascos aparecem parcialmente submersos.
- **Luz:** apenas duas — *HemisphereLight* (céu/reflexo do mar) e *DirectionalLight* (o sol, ou o luar no modo escuro). Antes eram quatro luzes, e cada luz aumenta o custo de cada pixel.
- **Névoa:** exponencial na cor do horizonte, com densidade dividida pelo afastamento da câmera para não "lavar" navios distantes.
- **Sombras:** somente no modal e na imagem grande da galeria (mapa de 1024²). Nos demais quadros, desligadas.

3.3 Ciclo de vida das cenas (o coração da otimização)

Cada `<canvas>` vira uma *entrada* no gerenciador, que decide **quando criar**, **quando desenhar** e **quando parar**:

Evento	Mecanismo	Efeito
Canvas chega a 120 px da tela	IntersectionObserver	A cena entra na fila de criação (no máximo uma por frame , para não travar a rolagem)
Canvas sai da tela	IntersectionObserver	Deixa de ser renderizada (o último quadro permanece visível)
Nenhuma cena visível	Loop verifica <code>active = 0</code>	O <code>requestAnimationFrame</code> é cancelado — zero trabalho
Aba do navegador oculta	<code>visibilitychange</code>	Loop parado; retoma ao voltar

Evento	Mecanismo	Efeito
Modal aberto	Flag <i>backgroundPaused</i>	Só a cena do modal renderiza; o fundo (coberto) pausa
Card esmaecido pelo filtro	Classe <i>is-dimmed</i>	A cena daquele card é pulada
Tamanho do quadro muda	ResizeObserver	setSize e projeção da câmera ajustados uma vez
v2.0 · Tema muda	Evento rn:themechange	setTheme() atualiza as cenas já criadas e redesenha as visíveis; as ainda não criadas já nascem no tema certo

```
function frame(now) {
  rafId = requestAnimationFrame(frame);
  const dt = Math.min((now - lastTime) / 1000, 0.05); // tempo real, limitado a 50 ms
  if (initQueue.length) initEntry(initQueue.shift()); // 1 cena nova por frame
  let active = 0;
  for (const e of entries) {
    if (!e.ready || !e.visible) continue;
    if (backgroundPaused && !e.isModal) continue;
    if (e.card && e.card.classList.contains('is-dimmed')) continue;
    if (e.resizePending && !applyResize(e)) continue;
    e.update(dt); e.renderer.render(e.scene, e.camera); active++;
  }
  if (!active && !initQueue.length) stop(); // nada a desenhar -> para o loop
}
```

3.4 Reaproveitamento de recursos

- **Cache de geometrias e materiais** (`G.box(...)`, `mat(color, opts)`): um mesmo par forma+materiais é criado *uma vez* e usado por todas as cenas e navios. Isso reduz memória e evita recompilar shaders. O material da água de cada tema também vem desse cache.
- **InstancedMesh**: peças repetidas (26 luzes do convés, 16 portinholas, 8 canhões, 4 tubos de torpedo, lemes, pás da hélice) viram *um* desenho cada, em vez de dezenas.
- **Renderer do modal criado uma vez** e reutilizado: ao trocar de navio, apenas o grupo 3D é substituído.
- **Canvas opaco** (`alpha:false`): o compositor do navegador não precisa misturar o canvas com o fundo da página.

3.5 Interação

Arrastar para girar usa **Pointer Events** com *pointer capture* (funciona igual para mouse, toque e caneta e não depende de listeners em window). Os canvases têm `touch-action: pan-y`: o gesto vertical continua rolando a página; o horizontal gira o navio. Depois de 2,5 s sem tocar, a rotação automática retorna. Toda a suavização usa $k = 1 - e^{-5 \cdot dt}$, portanto independe da taxa de quadros do monitor.

3.6 Enquadramento automático

Quadros “em pé” (como o painel esquerdo do modal, com proporção $\approx 0,85$) cortavam o navio. A câmera agora se afasta por $k = \sqrt{1,6 / \text{aspecto}}$ sempre que a proporção fica abaixo de 1,6, mantendo o navio inteiro visível.

3.7 Temas do 3D: dia de sol e noite de guerra (v2.0)

O que muda entre os modos não são os modelos, e sim o **cenário**. Um objeto `THEMES` descreve os dois, e cada cena guarda referências à luz hemisférica, à luz direcional e ao oceano para poder trocá-los sem recriar nada:

Parâmetro	Claro (dia de sol)	Escuro (noite de guerra)
Céu (4 paradas do gradiente)	#3d9fe4 → #86cdf3 → #cdeefb → #cdeefb	#02050b → #0b1c33 → #5a2a14 → #1a0d08 (horizonte em brasa)
Névoa (cor · fator de densidade)	0xcdeefb · ×1	0x140c10 · ×1,25
Água (cor · rugosidade · metalicidade)	0x0e8aa8 · 0,28 · 0,05	0x082733 · 0,22 · 0,25
Luz hemisférica (céu / “chão” · intensidade)	0xeaf6ff / 0xb4d2d6 · 0,66	0x5f80b0 / 0x8a3a14 · 0,95 — o chão alaranjado imita o reflexo dos incêndios
Luz direcional	0xffff0c2 · 1,0 (sol)	0xb4ceff · 1,15 (lunar)

- `setTheme(mode)` grava o tema corrente, aplica `applyTheme()` às cenas *já criadas* (fundo, cor da névoa, cores e intensidades das luzes, material da água) e chama `start()` para redesenhar as visíveis. Cenas ainda não criadas nascem no tema certo porque `createScene()` lê o tema corrente.
- A densidade da névoa é recalculada com `fogBase / fogScale`: o enquadramento automático (seção 3.6) afasta a câmera, e a névoa precisa acompanhar essa distância também no tema escuro (fator 1,25).
- As duas texturas de céu são criadas uma única vez (cache por tema): a segunda troca de tema não aloca nada.

4. O hero animado e as demais animações

4.1 Hero em Canvas 2D: um controlador, uma cena por tema

O fundo do hero é um `<canvas id="ocean-canvas">` 2D. Na v1.1 ele era desenhado por uma função única (`initHeroSea()`). Na v2.0 essa função foi dividida em duas partes, **sem mudar uma linha do desenho**: um **controlador** (`window.RNHero`) e a **cena clara**, registrada nele. A cena escura (`war-hero.js`) se registra do mesmo modo. O controlador cuida do que é comum; cada cena só sabe desenhar.

Responsabilidade	Quem faz	Como
Tamanho, DPR (limitado a 1,5) e canvas opaco	RNHero	<code>ResizeObserver</code> ; ao mudar o tamanho marca as cenas como “sujas” (só a ativa é recalculada na hora)
Pausa fora da tela e com a aba oculta	RNHero	<code>IntersectionObserver</code> no hero + <code>visibilitychange</code> ; sem loop, o quadro fica estático
Movimento reduzido	RNHero	Com <code>prefers-reduced-motion</code> o hero é desenhado uma vez, sem laço
Trocar de tema	<code>RNHero.setMode</code>	Escuta <code>rn:themechange</code> , troca a cena e pinta um quadro na hora (mesmo pausado)
Preparar o outro tema	RNHero	Quando o ponteiro entra no botão (<code>rn:prewarm</code>), a cena do outro tema já monta sprites e gradientes: a troca não engasga
Contrato de uma cena	cada cena	<code>{ resize(ctx, W, H, dpr), render(ctx, dt) }</code> — sem laço próprio, sem observers próprios

A cena clara — o dia ensolarado

O princípio é: **pré-renderizar tudo que não muda** e desenhar por frame só o que se move.

Elemento	Como é feito	Custo por frame
Céu + mar + névoa do horizonte	Pré-renderizados em um canvas fora da tela (refeito só ao redimensionar)	1 <code>drawImage</code>
Sol	Sprite radial pré-renderizado; leve pulsar por escala	1 <code>drawImage</code>

Elemento	Como é feito	Custo por frame
Nuvens (7)	3 sprites pré-renderizados, deriva horizontal por tempo	7 drawImage
Gaivotas (4)	Duas curvas quadráticas; batida de asa por seno	4 traçados curtos
Navio ao fundo e veleiro	Polígonos simples com balanço leve	Poucos preenchimentos
Ondas (5 camadas)	Senoides somadas, gradientes pré-calculados , brilho na crista	5 preenchimentos
Brilhos do sol na água (34)	Retângulos finos com cintilação individual	34 retângulos

- **Pausa inteligente:** a animação para quando o hero sai da tela ou a aba fica oculta (agora feito pelo RNHero, para as duas cenas).
- **Tempo real:** o movimento usa dt em segundos — um monitor de 144 Hz não anima mais 2,4× mais rápido que um de 60 Hz (como acontecia com $t += 0.016$ por frame).
- **Resolução:** DPR limitado a 1,5 e canvas opaco (`alpha:false`).
- **Quadro estático:** com `prefers-reduced-motion`, o hero é desenhado uma vez, sem loop.

4.2 Demais animações e como cada uma é feita

Animação	Implementação	Por que é fluida
Entrada do hero	CSS <code>@keyframes heroFadeIn</code> com atrasos escalonados	opacity + transform
Parallax do título	Um handler de scroll (rAF) escreve <code>translate3d</code> e opacidade	Sem leitura de layout no scroll; só roda com o hero visível
Navbar ao rolar	Classe <code>scrolled</code> (fundo, sombra e cores)	Transição só de cor/sombra — sem blur
Link ativo	IntersectionObserver com faixa no meio da tela	Nenhum trabalho no scroll
Contadores	rAF + <code>easeOutCubic</code> , 1,8 s, formato pt-BR	Baseado em tempo
Revelação ao rolar	IntersectionObserver adiciona <code>is-visible</code> ; ao fim da transição as classes são removidas	opacity/transform; o elemento volta ao CSS normal e o hover funciona
Hover dos cards	CSS <code>transform</code> + sombra	Compositor
Filtro de navios	Classe <code>is-dimmed</code> + atributo <code>inert</code>	Transição no CSS; itens fora do filtro saem do foco
Barras da frota	<code>scaleX(var(--fill))</code> com atraso escalonado	Compositor (antes animava <code>width</code>)
Modal	Fade do overlay + escala do contêiner; <code>visibility</code> escondido quando fechado	Sem blur; some do fluxo de acessibilidade ao fechar
Brilho do cursor	Um div movido por <code>translate3d</code> ; loop para ao alcançar o cursor	Só com mouse; sem <code>blend-mode</code>
v2.0 · Botão de tema	Trilho e botão em CSS: <code>transform</code> do “knob” (sol ↔ lua), estrelas que surgem no trilho	transform/opacity; sem JavaScript por quadro
v2.0 · Troca de tema	Revelação circular (View Transitions + clip-path) ou cortina circular de reserva	Feita pelo compositor; ver seção 5.3
v2.0 · Medidor de profundidade	Marcador por <code>translate3d</code> ; texto atualizado só quando o valor (a cada 10 m) muda	Sem layout por quadro

4.3 Acessibilidade incorporada

- `prefers-reduced-motion`: animações e transições praticamente desligadas; hero estático; sem parallax nem brilho; a troca de tema vira instantânea.
- Modal: foco vai para o botão fechar, **Tab** fica preso dentro do modal, **Esc** fecha e o foco volta ao elemento que abriu.
- Canvases 3D com `role="img"` e `aria-label`; canvas decorativo do hero e o fundo do mar com `aria-hidden`; as faixas submarinas têm `role="region"` e rótulo.
- Foco visível global (contorno âmbar), botões de filtro com `aria-pressed` e link ativo com `aria-current`.
- **Botão de tema**: `role="switch"` com `aria-checked` (verdadeiro no escuro), `aria-label` fixo, `title` que muda conforme o estado e rótulo visível "Claro/Escuro"; alcançável e acionável pelo teclado.

5. Modo escuro: guerra em alto mar (v2.0)

A versão 2.0 acrescenta um segundo modo ao site, escolhido por um botão no canto superior direito. O **modo claro** é o dia ensolarado de sempre — e permanece **exatamente** como era. O **modo escuro** transforma a página numa viagem da superfície ao leito do oceano, em uma noite de guerra. Esta seção documenta como isso foi construído, o que cada arquivo faz e por que a animação continua fluida.

5.1 A jornada: da superfície ao leito do mar

A ideia narrativa é que **rolar a página é afundar**. No topo, duas frotas trocam disparos sob uma tempestade; ao descer, a água escurece, surgem raios de luz, neve marinha e cardumes; no meio do caminho há uma faixa de combate submarino; e, no fim, o visitante chega aos navios que a guerra levou. Um medidor de profundidade acompanha a descida.

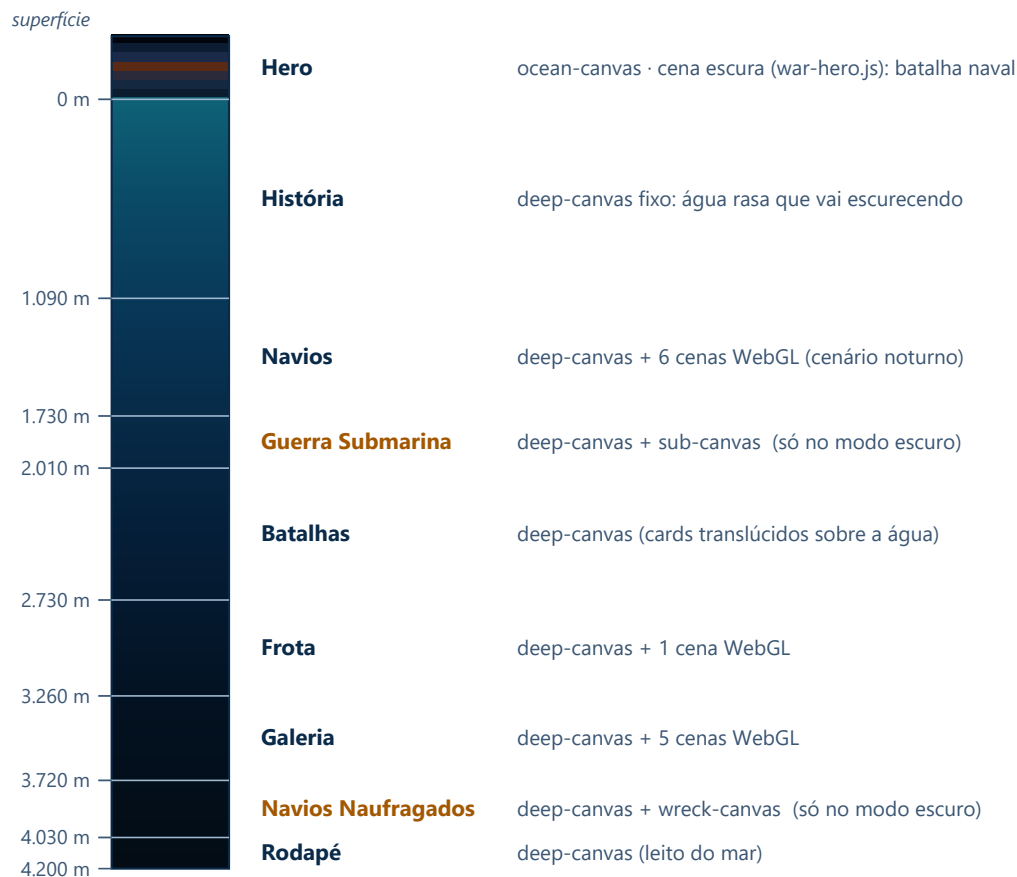


Figura 7 — Mapa da página no modo escuro (proporções reais, medidas a 1366×768: 10.294 px de altura; 8.941 px no modo claro). A cor da barra é a cor da água que o canvas fixo desenha em cada profundidade. Os valores à esquerda são a profundidade mostrada pelo medidor quando o *centro da tela* está naquele ponto. Em laranja, as duas faixas que só existem no modo escuro.

Trecho	Profundidade mostrada	O que o visitante vê	Onde está no código
Hero	— (medidor oculto)	Batalha naval noturna: 4 navios trocam tiros; explosões, incêndios, relâmpagos, holofotes; navios afundam e voltam	war-hero.js
História	≈ 170 → 920 m	Água turquesa-escura com raios de luz, neve marinha, bolhas, cardumes e submarinos distantes	deep-sea.js (Deep)
Navios	≈ 1.250 → 1.560 m	Os 6 modelos 3D sob luar; a água já é azul-marinho	ships-3d.js + Deep
Guerra Submarina	≈ 1.870 m	Dois submarinos (verde-água × vermelho) trocam torpedos; sonares; cargas de profundidade; um terceiro observa ao fundo	deep-sea.js (subWar)
Batalhas	≈ 2.180 → 2.560 m	Cards translúcidos sobre a água; sem as ondas claras do modo claro	dark.css + Deep
Frota · Galeria	≈ 2.900 → 3.550 m	Cenas 3D em formação; criaturas luminosas surgem nas profundezas	ships-3d.js + Deep
Navios Naufragados	≈ 3.870 m	Destroieres partidos, um galeão, um submarino e um porta-aviões no leito; algas, corais, cardumes, baliza de socorro	deep-sea.js (wrecks)
Rodapé	até ≈ 4.030 m	Leito do mar	dark.css

A profundidade é **narrativa**: vai de 0 a 4.200 m em proporção à rolagem (o medidor usa o centro da tela; no fim da página marca ≈ 4.030 m). As zonas seguem os limites oceanográficos usuais (epipelágica até 200 m, mesopelágica até 1.000 m,

batipelágica até 4.000 m, abissal a partir daí).

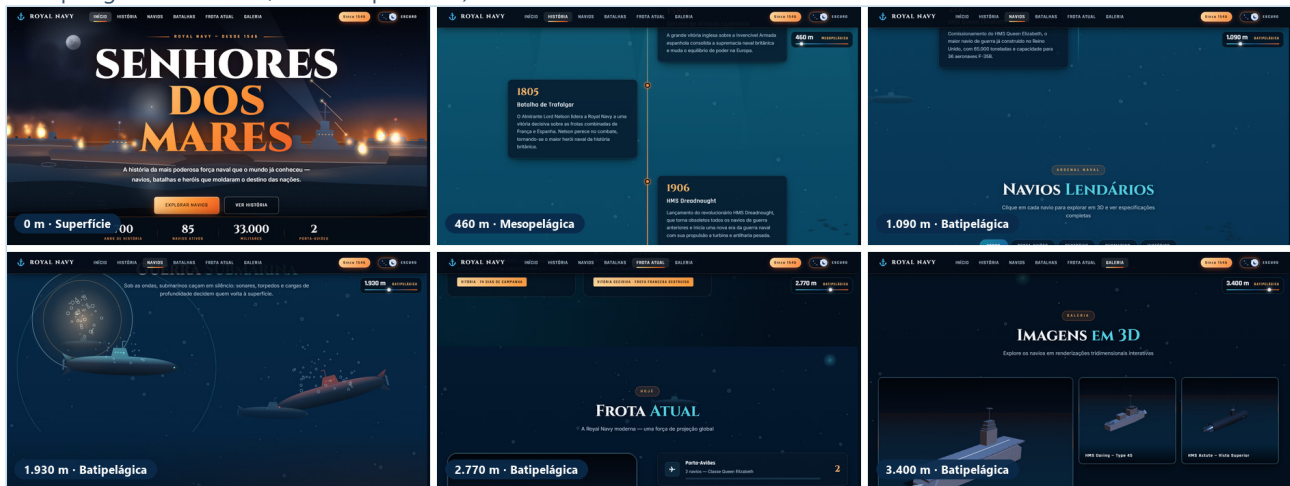


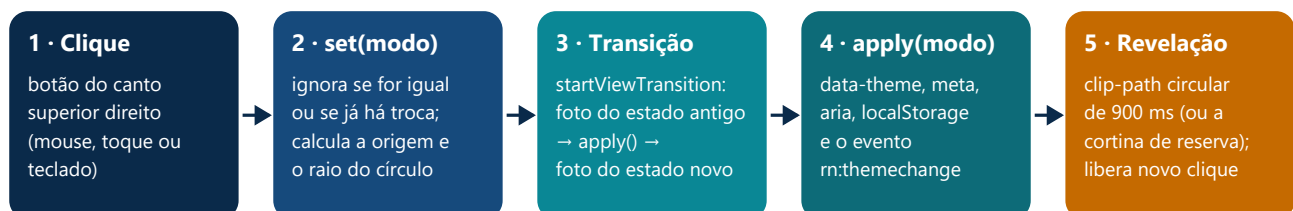
Figura 8 — O mergulho em seis pontos da página (modo escuro, 1366×768): da batalha na superfície até 3.400 m (batipelágica). O medidor à direita mostra metros e zona.

5.2 Como o tema funciona

O tema é um **atributo no elemento raiz**: `<html data-theme="light">` ou `<html data-theme="dark">`. Tudo que é específico do escuro se ativa por esse atributo — no CSS, com seletores `[data-theme="dark"]` ...; no JavaScript, lendo o atributo ou escutando um evento. Quatro peças cooperam:

Peça	Arquivo	Papel
Atributo <code>data-theme</code>	<code><html></code>	Fonte única da verdade sobre o tema atual. O padrão é <code>light</code> .
Aplicação inicial	<code>theme-init.js</code> (11 linhas)	Script <i>síncrono</i> no <code><head></code> : lê <code>localStorage['rn-theme']</code> , define <code>data-theme</code> e a cor da barra do navegador (meta <code>theme-color</code>) antes da primeira pintura — sem “pisca” claro→escuro. É um arquivo externo (e não inline) por causa da CSP.
Controle	<code>theme.js</code> (140 linhas)	<code>window.RNTheme = { get, set, toggle }</code> : valida, anima a troca, atualiza botão e armazenamento e dispara o evento <code>rn:themechange</code> . Também abriga o <code>RNPerf</code> (seção 5.8).
Reações	<code>main.js</code> · <code>deep-sea.js</code> · <code>ships-3d.js</code> · CSS	Cada módulo escuta o evento (ou apenas o atributo) e faz só o seu trabalho — não há um “gerente de tema” que conheça todos.

O que acontece ao clicar



Ouvintes do evento `rn:themechange` (todos reagem no mesmo quadro, sem recarregar nada):

RNHero troca a cena do hero	ShipEngine luz/céu/água do 3D	Deep refaz a medição do fundo	band() × 2 faixas submarinas	RNPerf liga/desliga o medidor
---------------------------------------	---	---	--	---

Figura 9 — Sequência de uma troca de tema. O mesmo caminho vale para o clique, o teclado e para chamadas programáticas (`RNTheme.set('dark')`).

Ouvinte	Arquivo	O que faz ao receber <code>rn:themechange</code>
<code>RNHero.setMode</code>	<code>main.js</code>	Troca a cena do hero (clara ↔ escura) e pinta um quadro na hora, mesmo com o hero pausado
<code>ShipEngine.setTheme</code>	<code>ships-3d.js</code>	Atualiza céu, névoa, luzes e água das cenas 3D já criadas e redesenha as visíveis
<code>Deep (remeasure)</code>	<code>deep-sea.js</code>	No escuro: mede a altura da página e do hero e liga o laço do fundo se houver rolagem. No claro: só desliga
<code>band() × 2</code>	<code>deep-sea.js</code>	Recalcula o tamanho dos canvases das faixas (que passam de <code>display:none</code> a visíveis) e liga/desliga o laço
<code>RNPerf (syncPerf)</code>	<code>theme.js</code>	Liga o medidor de tempo de quadro só enquanto o tema é escuro
CSS	<code>style.css · dark.css</code>	Seletores por atributo: cores, sombras, fundos, botões, medidor, faixas — tudo muda sem uma linha de JavaScript

Persistência e padrões

- A escolha fica em `localStorage[ 'rn-theme' ]` ('light' ou 'dark'). Toda leitura/gravação está em `try/catch`: em janela privada ou com o armazenamento bloqueado o site simplesmente abre no claro.
- O padrão é **sempre o claro**; a preferência do sistema operacional não é consultada (ver decisões, seção 2.6).
- A cor da barra do navegador no celular (`theme-color`) acompanha: `#2a9bdc` no claro, `#050b14` no escuro.
- O botão reflete o estado em `aria-checked`, no `title` ("Ativar o modo escuro (guerra no mar)" / "Voltar ao modo claro") e no rótulo "Claro/Escuro".

```
// theme.js – RNTheme.apply(): o único lugar que muda o tema
function apply(mode) {
  root.setAttribute('data-theme', mode); // 1) CSS reage sozinho
  if (meta) meta.setAttribute('content', COLORS[mode]); // 2) cor da barra do navegador
  if (btn) { btn.setAttribute('aria-checked', String(mode === 'dark')); btn.title = /* ... */; }
  if (label) label.textContent = mode === 'dark' ? 'Escuro' : 'Claro';
  try { localStorage.setItem(KEY, mode); } catch (e) { /* ignora */ }
  document.dispatchEvent(new CustomEvent('rn:themechange', { detail: { theme: mode } })); // 3) avisa os módulos
}
```

5.3 A animação de transição e o botão

Revelação circular. Ao clicar, o novo tema “nasce” do botão e se expande em círculo até cobrir a tela. Isso usa a **View Transitions API** do navegador: ele tira uma “foto” da página no estado antigo, executa a troca de tema e tira uma “foto” do estado novo; as duas ficam empilhadas como pseudo-elementos `::view-transition-old(root)` e `::view-transition-new(root)`. O site desliga a animação padrão (um esmaecimento) e anima só a camada nova com `clip-path: circle()` — de raio 0, no centro do botão, até o raio que alcança o canto mais distante da tela — durante **900 ms** com `cubic-bezier(.4, 0, .2, 1)`. Quem faz o trabalho é o compositor do navegador; o JavaScript apenas dispara a animação.



Figura 10 — A revelação circular congelada em quatro instantes (animação de 900 ms; o tempo é o da animação, com o easing aplicado). O modo escuro nasce no canto do botão e a batalha já anima dentro do círculo.

```
// theme.js - RNTheme.set() (trecho simplificado)
function set(mode, opts = {}) {
  if (mode !== 'dark' && mode !== 'light') return;
  if (mode === get() || busy) return; // igual ao atual, ou troca em andamento
  if (opts.instant || REDUCED || !btn) { apply(mode); return; }

  const r = btn.getBoundingClientRect();
  const x = r.left + r.width / 2, y = r.top + r.height / 2; // origem = centro do botão
  const radius = Math.hypot(Math.max(x, innerWidth - x), Math.max(y, innerHeight - y)); // até o canto mais longe

  busy = true;
  if (typeof document.startViewTransition === 'function') {
    root.classList.add('theme-switching');
    const vt = document.startViewTransition(() => apply(mode)); // foto antiga → troca → foto nova
    vt.ready.then(() => root.animate(
      { clipPath: [`circle(0px at ${x}px ${y}px)`, `circle(${radius}px at ${x}px ${y}px)` ] },
      { duration: 900, easing: 'cubic-bezier(.4,0,.2,1)', pseudoElement: '::view-transition-new(root)' });
    vt.finished.then(finish, finish); // libera novo clique
  } else {
    veilTransition(mode, x, y, radius, finish); // cortina circular de reserva
  }
}
```

Situação	Comportamento
Navegador com View Transitions	Revelação circular acima. Enquanto dura, <html> recebe a classe theme-switching e novos cliques são ignorados (flag busy).
Navegador sem a API	Cortina de reserva: um <div class="theme-veil"> (cor do tema de destino) cresce em círculo por 520 ms, o tema é aplicado quando a tela está coberta e a cortina se desfaz em 380 ms. Testada simulando a ausência da API.
<i>prefers-reduced-motion</i>	Sem animação: a troca é instantânea.
Chamada programática	<code>RNTheme.set('dark', { instant: true })</code> aplica na hora (usado nos testes).
Mouse chega ao botão	Evento único <code>rn:prewarm</code> (também com foco por teclado ou toque): o RNHero já monta os sprites e gradientes da cena do <i>outro</i> tema, para a troca não engasgar.

O botão

O botão é um `<button role="switch">` com trilho e “bolinha” desenhados em CSS: no claro o trilho é azul-céu e a bolinha é um **sol** à esquerda; no escuro o trilho vira azul-noite com borda e brilho cor de brasa, surgem **estrelas** no trilho e a bolinha desliza 32 px para a direita transformando-se em **lua** (os dois ícones SVG trocam de opacidade, giro e escala). Tudo por `transform` e `opacity`. Abaixo de 1100 px de largura o rótulo “Claro/Escuro” some e só o botão fica; no celular ele vai para a extremidade direita da barra.

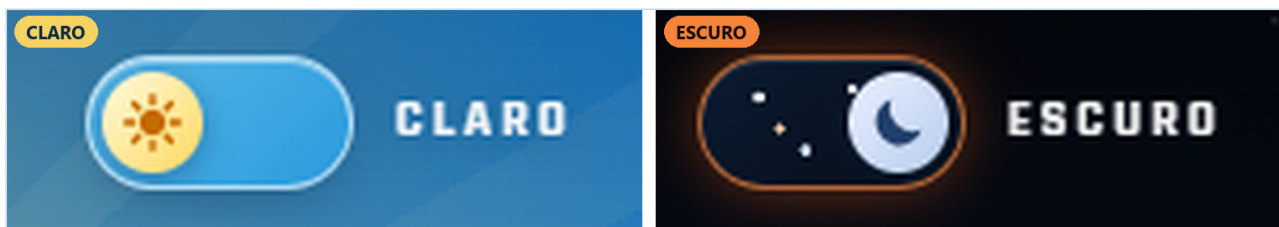


Figura 11 — O botão de tema ampliado 5× nos dois estados (recortes das capturas do hero).

5.4 O CSS do modo escuro (dark.css)

Todo o visual do modo escuro está em **um arquivo**, `dark.css`, cujos seletores começam por `[data-theme="dark"]` (ou são classes que só existem no escuro). O `style.css` ganhou **116 linhas novas e nenhuma alterada** — botão, transição e a regra que oculta, no modo claro, os elementos exclusivos do escuro. É por isso que o claro continua idêntico (comprovado na seção 9.5).

Bloco do <code>dark.css</code>	O que define
Tokens (<code>:root[data-theme="dark"]</code>)	Variáveis: <code>--scene-bg</code> (fundo das cenas 3D), <code>--grad-fire</code> (gradiente de brasa nos títulos), <code>--d-ember #ff8a3d</code> , <code>--d-ember-1 #ffb86b</code> , <code>--d-cyan #54d1e4</code> , painéis <code>--d-panel</code> translúcidos, textos <code>--d-tx/-tx2/-tx3</code> , sombras. Também <code>color-scheme: dark</code> (barras de rolagem e controles nativos escuros).
Base e componentes	Corpo, seleção, foco, barra de rolagem, botões, filtro, navbar (rolada), selo, cards de linha do tempo/navios/batalhas/frota/galeria, modal, rodapé, brilho do cursor (vira clarão de incêndio).
Hero	Fundo de noite, vinheta central reduzida (para a batalha aparecer), brilho de incêndio no horizonte (<code>fireGlow</code> : só a opacidade anima), título com gradiente de brasa, barra de estatísticas escura.
Seções sobre o mar	História, Navios, Frota e Galeria ficam com <code>background: transparent</code> : quem dá a cor é o canvas fixo do mar profundo; os cards usam painéis translúcidos por cima.
Faixas submarinas	<code>.abyss-band</code> (altura <code>clamp(440px, 84vh, 780px)</code> ; naufrágios <code>clamp(520px, 92vh, 860px)</code>), legenda sobre o canvas e um degradê que encosta o fundo no rodapé.
Medidor de profundidade	HUD fixo (top: 82px; right: 16px; 74/10 px no celular), <code>pointer-events: none</code> , entra com fade e desliza; marcador movido por <code>translate3d</code> .
Batalhas e rodapé	As ondas divisórias claras somem (<code>display: none</code>) e o fundo vira um degradê escuro translúcido.

No modo claro, três elementos existem no HTML mas ficam ocultos por `.deep-canvas`, `.depth-gauge`, `.abyss-band { display: none }` (regra em `style.css`): o canvas do mar profundo, o medidor e as duas faixas. Como `display:none` zera o tamanho, os canvases nem chegam a ser dimensionados nem a executar laços.

5.5 O hero em batalha (war-hero.js)

A cena escura do hero é **Canvas 2D** puro (741 linhas). Ela se registra no controlador com `RNHero.register('dark', createWarScene())` e obedece ao contrato `{ resize(ctx, W, H, dpr), render(ctx, dt) }`: não tem laço, observers nem listeners próprios — o controlador cuida de pausa, visibilidade e tamanho.

Os atores

Navio	Lado	Posição (x)	Comprimento	Intervalo entre salvas	Acertos p/ afundar
Porta-aviões	1 (luz azul)	7,5%	22% da largura	6–9 s (1 tiro)	9
Destróier	1 (luz azul)	26,5%	33%	1,2–2,2 s (até 2 tiros)	7
Couraçado	2 (luz vermelha)	73,5%	35%	1,4–2,4 s (até 2 tiros)	8
Cruzador	2 (luz vermelha)	92,5%	21%	2,4–4 s (até 2 tiros)	6

Porta-aviões e cruzador ficam *ao fundo* (desenhados atrás das duas primeiras camadas de onda); destróier e couraçado, à frente. Em telas estreitas (largura < 600 px) os navios são ampliados $\times 1,34$.

O ciclo de combate

- **Disparo.** Cada navio sorteia um inimigo ainda ativo e um ponto do casco; sai um *clarão de boca* (branco + laranja), fumaça e o projétil parte em **arco parabólico** (0,85–1,35 s). Há 55% de chance de acerto; se errar, cai na água perto do alvo.
- **Acerto.** Explosão (35% delas “grandes”) com clarões, chamas, faíscas e fumaça; o alvo soma um acerto e ganha um **incêndio a bordo** (até 4 por navio, 7–12 s cada) que continua soltando chamas e fumaça. **Erro:** anel de onda + coluna d’água (gotas) + clarão azulado.
- **Naufrágio.** Ao atingir o limite de acertos o navio *afunda em 9 s* (desce 15% do comprimento, inclina até 0,4 rad, some no fim, soltando fumaça); depois fica *ausente 4,5 s* e **volta ao combate** com um esmaecimento de 2 s. A batalha nunca acaba.
- **Atmosfera.** Relâmpago a cada 4–9 s (traço irregular com ramificação e clarão no céu); *sinizador de iluminação* a cada 7–11 s (sobe em 1 s e desce em 8 s, iluminando a cena e o mar); 2–3 **holofotes** que varrem o céu; 16 brilhos da lua na água; clarões distantes no horizonte; um “clarão geral” que decai (até 0,3) a cada tiro e explosão.

```
// war-hero.js – máquina de estados de cada navio (trecho simplificado de update())
if (s.state === 'ok') {
  s.cool -= dt;
  if (s.cool <= 0) { s.cool = rnd(s.rate[0], s.rate[1]); fire(s); } // atira
  if (s.hits >= s.dmgMax) { s.state = 'sinking'; s.sinkT = 0; }
} else if (s.state === 'sinking') {
  s.sinkT = Math.min(1, s.sinkT + dt / 9); // 9 s afundando
  s.sinkY = s.sinkT * s.L * 0.15; sinkRoll = s.sinkT * 0.4 * s.dir;
  s.alpha = 1 - clamp((s.sinkT - 0.78) / 0.22, 0, 1); // some no fim
  if (s.sinkT >= 1) { s.state = 'gone'; s.gone = 4.5; ring(s.x, s.y); }
} else if (s.state === 'gone') {
  s.gone -= dt;
  if (s.gone <= 0) { s.state = 'ok'; s.hits = 0; s.alpha = 0; s.fade = 1; } // volta ao combate
}
```



Figura 12 — Três quadros da batalha (recortes 1366×440, alguns segundos entre eles): incêndios a bordo, projéteis em arco, traçantes, clarões e holofotes.

Como a cena custa pouco

Técnica	Onde	Efeito
Pré-renderização	céu com 90 estrelas, lua com halo e crateras, incêndios do horizonte, mar; silhuetas dos navios; sprites de brilho, fumaça, feixe e nuvens de tempestade	Feita <i>uma vez</i> por redimensionamento; por quadro é só drawImage.
Silhuetas com luz	cada navio é um sprite com luar por cima e brilho de incêndio por baixo, aplicados com source-atop (só nos pixels já desenhados)	Aparência de iluminação sem custo por quadro.
Brilhos aditivos	explosões, chamas, clarões, projéteis, luzes de navegação (globalCompositeOperation = 'lighter')	Sprites radiais reaproveitados em vez de gradientes criados a cada frame.
Pools de tamanho fixo	partículas 150 · clarões 28 · anéis 10 · projéteis 26 · traçantes 18	Nenhuma alocação por quadro: sem pausas do coletor de lixo.
Ondas	5 camadas com gradientes pré-calculados (um por camada)	Senoides somadas; 5 preenchimentos por quadro.
Tempo real	tudo por dt (limitado a 50 ms)	Mesma velocidade em 60 Hz ou 144 Hz.
Escala	S = clamp(W / 1366, 0.6, 1.35)	Tamanhos, velocidades e efeitos proporcionais à tela.

```
// war-hero.js – partículas em pool: nunca cria objetos durante a batalha
const CAP = [44, 84, 150]; // teto ativo por nível do RNPerf
const P = Array.from({ length: 150 }, () => ({ a: 0, type: 0, x: 0, y: 0, vx: 0, vy: 0, life: 0, max: 1, size: 10, y0: 0 }));
function spawn(type, x, y, vx, vy, life, size, y0) {
  const cap = CAP[level()];
  for (let k = 0; k < cap; k++) {
    pIdx = (pIdx + 1) % cap;
    const p = P[pIdx];
    if (!p.a) { p.a = 1; p.type = type; p.x = x; p.y = y; /* ... */ return p; }
  }
  return null; // sem vaga: descarta em vez de alocar
}
```

5.6 O mergulho, a guerra submarina e os naufrágios (deep-sea.js)

O segundo arquivo do modo escuro (791 linhas) tem quatro módulos independentes. Nenhum usa laço próprio quando está invisível.

Módulo	Canvas / elemento	Função
Gauge	#depth-gauge (HTML)	Converte a rolagem em metros e zona; move o marcador
Deep	#deep-canvas (fixo, atrás de tudo)	A água: cor por profundidade, superfície, raios de luz, neve, bolhas, cardumes, submarinos distantes, criaturas luminosas
subWar	#sub-canvas (faixa "Guerra Submarina")	Submarinos, torpedos, sonares, cargas de profundidade, explosões
wrecks	#wreck-canvas (faixa "Navios Naufragados")	Naufrágios no leito, algas, cardumes, bolhas, baliza de socorro

O fundo do mar (Deep)

Um único <canvas> **fixo**, atrás do conteúdo, desenhado em **meia resolução** (SC = 0.5: um quarto dos pixels) e opaco. A profundidade de cada ponto da página é a fração $(y - \text{altura do hero}) \div (\text{altura da página} - \text{altura do hero})$; a cor da água é interpolada entre seis paradas; cada quadro pinta um degradê vertical que cobre exatamente o trecho da página visível naquele instante.

Profundidade (fração)	Cor da água	Elementos ativos
0 (logo abaixo do hero)	rgb(14, 98, 118) · #0e6276	Linha d'água ondulante e 4 raios de luz (força $\propto (1 - d)^{1,7}$); cardumes ($d < 0,55$)
0,12	rgb(10, 76, 104) · #0a4c68	Neve marinha (opacidade $0,28 + 0,4 \cdot d$) e bolhas subindo, com paralaxe
0,30	rgb(8, 52, 84) · #083454	3 submarinos distantes (28% de opacidade) com clarões e ondas de choque de combates ao longe, a cada 5–10 s
0,55	rgb(5, 31, 58) · #051f3a	Criaturas luminosas (7) surgem a partir de $d > 0,35$, com brilho ciano ou verde
0,80	rgb(3, 17, 32) · #031120	Neve mais densa; quase sem luz
1 (fim da página)	rgb(3, 12, 20) · #030c14	Leito

```
// deep-sea.js — a profundidade é a posição na página
const dAt = y => clamp((y - heroH) / Math.max(1, docH - heroH), 0, 1); // 0 = abaixo do hero · 1 = fim
const d0 = dAt(scrollY), d1 = dAt(scrollY + innerHeight);
for (let i = 0; i <= 4; i++) g.addColorStop(i / 4, colAt(d0 + (d1 - d0) * (i / 4))); // água entre topo e base da tela

// só roda quando faz sentido:
const want = isDark() && !document.hidden && window.scrollY > 2; // no hero (scrollY=0) ele nem desenha
```

O medidor de profundidade

O valor em metros é $\text{round}(d \times 4200 / 10) \times 10$ com d calculado no *centro* da tela; o texto só é reescrito quando muda (a cada 10 m) e o marcador se move por `translate3d`. As zonas do medidor são: 0 m Superfície · 30 m Epipelágica · 200 m Mesopelágica · 1.000 m Batipelágica · 4.000 m Abissal. O HUD só aparece depois que a página desce mais de 40% da altura da tela.

Faixas com canvas próprio: o utilitário band()

As duas faixas usam a mesma função auxiliar, que garante que o canvas **só anima quando precisa**: com tema escuro, faixa a menos de 80 px da tela, aba visível e sem *reduced-motion*. O canvas é transparente (a água é a do fundo fixo) e usa DPR limitado a 1,25.

```
// deep-sea.js — band(): o laço só existe enquanto o canvas está visível no modo escuro
function sync() {
```

```
const want = isDark() && visible && !document.hidden && !REDUCED;
if (want && !raf) { last = performance.now(); raf = requestAnimationFrame(frame); }
else if (!want && raf) { cancelAnimationFrame(raf); raf = 0; }
}
new IntersectionObserver([e] => { visible = e.isIntersecting; sync(); }, { rootMargin: '80px 0px' }).observe(box);
new ResizeObserver(resize).observe(canvas); // no claro o canvas tem tamanho 0: resize() sai logo
```

Guerra Submarina (subWar)

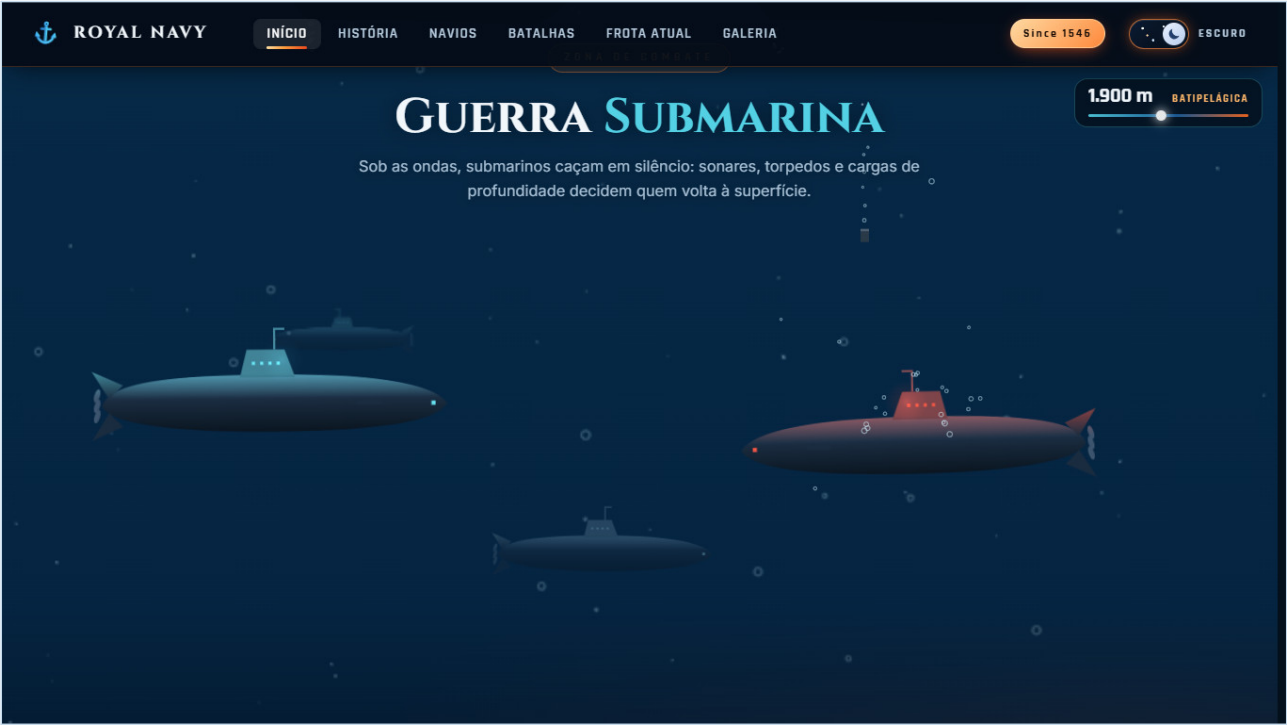


Figura 13 — Faixa “Guerra Submarina”: submarino verde-água (esquerda) e vermelho (direita) em combate, um terceiro ao fundo; bolhas, sonar e o medidor em 1.900 m.

Elemento	Regra
Submarinos	2 combatentes (verde-água, sentido → · vermelho, sentido ←) e 1 observador distante (42% de opacidade, não combate); cada um oscila lentamente e tem 2 pontos de vida
Torpedos	Saem a cada 2,3–3,8 s de cada combatente, a 0,55 largura/s, com esteira de bolhas; 55% acertam; erros passam perto e dão clarão + anel
Sonar	A cada 3,2–4,4 s cada combatente emite um anel que cresce até 42% da largura
Cargas de profundidade	A cada 3,2–5,5 s uma carga cai de um dos 2 cascos de superfície (70–100 px/s) e explode entre 42% e 74% da altura: clarão, 2 ondas de choque, bolhas, destroços e tremor da cena
Avaria e naufrágio	Um acerto faz o submarino vazar bolhas por 6 s; o segundo o afunda (3,4 s, inclinando e esmaecendo); após 2,6 s ausente ele reaparece — como na batalha da superfície, o combate é contínuo
Pools	torpedos 8 · bolhas 150 (teto 60/100/150 pelo RNPerf) · anéis 18 · clarões 10 · cargas 6 · destroços 30

Navios Naufragados (wrecks)



Figura 14 — Faixa “Navios Naufragados”: cascos partidos no leito, algas, cardumes e a baliza de socorro (ponto vermelho); o medidor marca 3.890 m (zona abissal).

Elemento	Como é feito
Naufrágios (5 sprites)	Desenhados por polígonos quebrados, com cavernas, ferrugem e corais aplicados por source-atop e “névoa” de profundidade: porta-aviões (ao fundo, 50% de opacidade), proa e popa de destróier, galeão (inclinado 0,34 rad, mastros e velas rasgadas) e submarino
Aleatoriedade repetível	Um gerador pseudoaleatório com <i>semente fixa</i> por sprite: o desenho é sempre o mesmo a cada carregamento
Leito	Areia com pedras e conchas, uma âncora e canhões espalhados, pré-renderizados uma vez e desenhados <i>na frente</i> da base dos cascos
Vida	20+ algas (2 traçados no total, um por cor) que balançam; 2 cardumes; bolhas saindo de 3 pontos dos cascos; neve marinha; 3 raios de luz muito fracos
Baliza	Luz vermelha piscando no destróier, com um anel de sonar a cada 4,2 s

5.7 O 3D em cenário noturno

Os modelos são os mesmos; o que muda é o cenário (lunar, névoa escura, água escura, reflexo alaranjado dos incêndios), como detalhado na seção 3.7. Os painéis, o modal e os rótulos ganham o visual escuro por CSS.

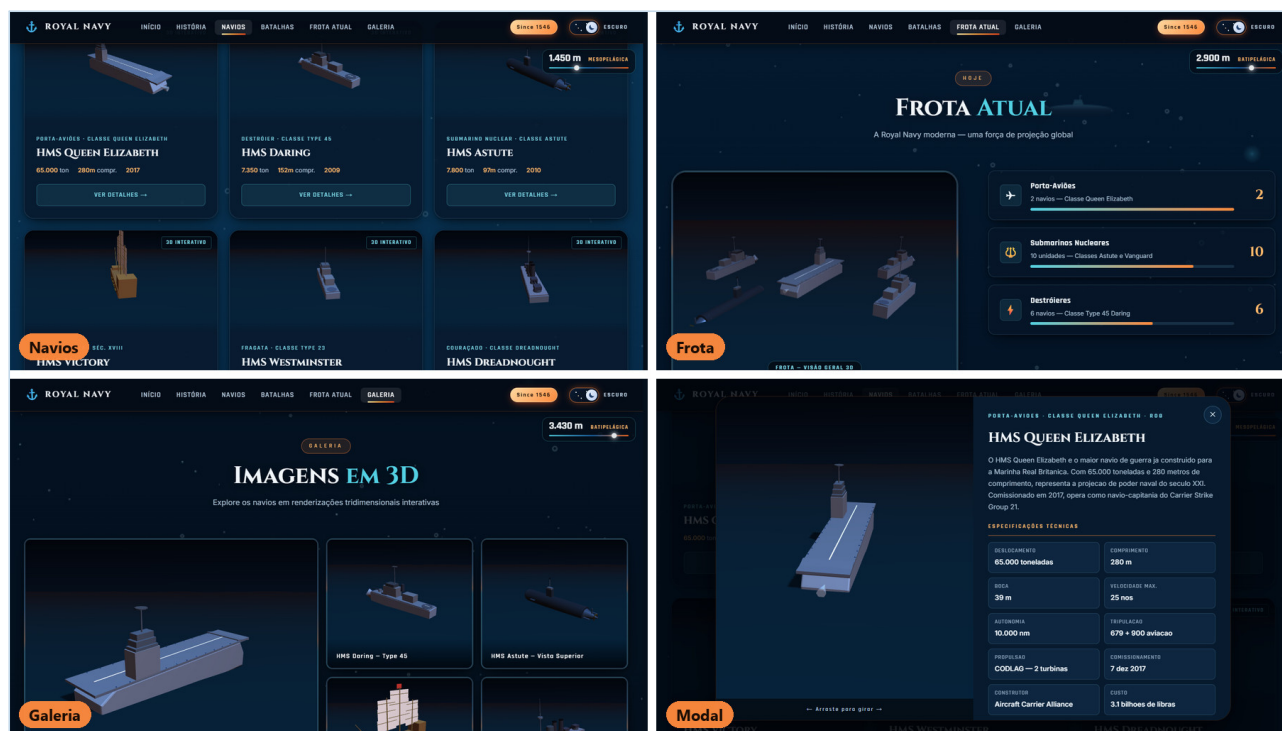


Figura 15 — Demais seções no modo escuro (1366×768, reduzidas): grade de navios, frota, galeria e o modal do HMS Queen Elizabeth.

5.8 Desempenho do modo escuro

O modo escuro tem muito mais coisas em movimento que o claro — e mesmo assim precisa continuar fluido. A regra de ouro é: **nada roda se não estiver sendo visto**.

Canvas	Roda quando...	Resolução	Observação
ocean-canvas (hero)	hero visível · aba visível · sem reduced-motion (a cena depende do tema)	CSS × DPR ($\leq 1,5$)	Um só laço para as duas cenas
deep-canvas (fundo)	tema escuro · aba visível · rolagem > 2 px	50% (meia resolução)	No claro nem mede o layout (display:none)
sub-canvas	tema escuro · faixa a ≤ 80 px da tela · aba visível	CSS × DPR ($\leq 1,25$)	Transparente sobre o fundo
wreck-canvas	tema escuro · faixa a ≤ 80 px da tela · aba visível	CSS × DPR ($\leq 1,25$)	Transparente sobre o fundo
Amostrador RNPerf	tema escuro · aba visível	—	Um requestAnimationFrame leve
Cenas 3D	regras da seção 3.3 (inalteradas)	—	Trocar de tema não recria nada

Técnicas usadas

- **Pré-renderizar o que não muda** (céu, mar, silhuetas, brilhos, leito, corais) e desenhar por quadro só o que se move.
- **Sprites em vez de gradientes por quadro**: brilhos, fumaça e feixes são imagens radiais criadas uma vez e reutilizadas (com mistura aditiva).
- **Pools de tamanho fixo** para partículas, clarões, anéis, torpedos, bolhas e destroços: zero alocação por quadro.
- **Meia resolução** para o fundo fixo: a cor e as partículas suaves não precisam de todos os pixels.
- **Só o necessário no modo claro**: elementos do escuro com display:none

- **Tempo real (dt)** e teto de 50 ms por quadro; DPR limitado; canvases opacos quando possível; um único traçado para todas as bolhas e outro por cor de alga.
- **Compositor:** o medidor e o botão só animam transform e opacity; o brilho de incêndio do hero anima só a opacidade.
- **Pré-aquecimento:** a cena do outro tema é preparada quando o mouse chega ao botão.

Qualidade adaptativa: RNPerf

Máquinas diferentes têm capacidades diferentes. O RNPerf (em theme.js) mede o tempo médio de quadro em janelas de 45 quadros, **somente no modo escuro**, e ajusta um nível de qualidade RNPerf.level (2 alto, 1 médio, 0 baixo) que as cenas consultam a cada quadro:

Regra	Valor
Descer um nível	média da janela > 26 ms (menos de ≈ 38 fps), nível > 0 e fora do período de espera (2,5 s)
Subir um nível	média < 18,5 ms (≈ 54+ fps) por 14 s seguidos, nível < 2 e fora da espera (4 s)
Ignorado	quadros > 250 ms (aba voltou do segundo plano)

Efeito	Nível 2 (alto)	Nível 1 (médio)	Nível 0 (baixo)
Hero: partículas ativas	150	84	44
Hero: explosões e respingos (fator)	1,0	0,8	0,5
Hero: fogo e fumaça a bordo (fator)	1,0	0,65	0,35
Hero: traçantes · sinalizador de iluminação	sim · sim	sim · sim	não · não
Hero: holofotes · brilhos da lua · clarão geral	sim	sim	não
Hero: nuvens de tempestade	5	5	3
Mar profundo / naufrágios: neve e bolhas	100%	70%	40%
Mar profundo / naufrágios: raios de luz e cardumes	sim	sim	não
Submarinos: teto de bolhas · esteira dos cascos	150 · sim	100 · sim	60 · não

```
// theme.js - RNPerf (simplificado): janelas de 45 quadros, histerese e período de espera
function sample(now) {
  raf = requestAnimationFrame(sample);
  const dt = now - last; last = now;
  if (dt > 250) return; // aba voltou do segundo plano: ignora
  acc += dt; if (++n < 45) return;
  const avg = acc / n; acc = 0; n = 0;
  cooldown = Math.max(0, cooldown - 45 * avg);
  if (avg > 26 && Perf.level > 0 && cooldown === 0) { Perf.level--; cooldown = 2500; calmMs = 0; } // lento
  else if (avg < 18.5) { calmMs += 45 * avg; // folgado
    if (calmMs > 14000 && Perf.level < 2 && cooldown === 0) { Perf.level++; cooldown = 4000; calmMs = 0; } }
  else calmMs = 0;
}
```

Custo de carga

Arquivo novo	Bruto	Com gzip*	Como carrega
theme-init.js	0,5 KB	0,4 KB	síncrono, no head (11 linhas)
dark.css	18,4 KB	4,2 KB	no head; bloqueia a pintura, como todo CSS
theme.js	6,1 KB	2,4 KB	defer
war-hero.js	33,7 KB	9,7 KB	defer
deep-sea.js	43,3 KB	12,6 KB	defer
Total dos 5 arquivos	102,0 KB	29,3 KB	
Crescimento dos arquivos existentes	+ 12,4 KB	+ 3,3 KB	style.css, main.js, ships-3d.js e index.html

* Estimado com `gzip -9` local; o GitHub Pages usa a sua própria compressão. Soma dos arquivos servidos (sem o `three.js`, que vem do CDN): $\approx 34,7 \text{ KB} \rightarrow \approx 67,3 \text{ KB}$ com `gzip`. Esse custo existe mesmo para quem nunca ativa o modo escuro. Resultados medidos de FPS: seção 9.3.

5.9 Acessibilidade, segurança e compatibilidade

- **CSP intacta:** a política não foi afrouxada. Como ela proíbe scripts inline, o aplicador de tema é um arquivo externo (`theme-init.js`); nenhuma origem externa nova foi adicionada; a validação com o domínio Log do DevTools não registrou violações.
- **Botão acessível:** `<button role="switch">` com `aria-checked`, foco visível (contorno âmbar) e acionável por teclado.
- **Canvases decorativos** com `aria-hidden` (fundo do mar, medidor); faixas com `role="region"` e `aria-label`; canvases das faixas com `role="img"` e descrição.
- **Movimento reduzido:** com `prefers-reduced-motion` não há laços (o hero e o fundo são desenhados como quadro estático), a troca de tema é instantânea e as animações CSS ficam praticamente desligadas.
- **Contraste no escuro:** textos entre 6,5:1 e 17,4:1 sobre o pior painel (medido, seção 8).
- **Compatibilidade:** o suporte à View Transitions API varia por navegador e versão; por isso existe a cortina de reserva. Os testes foram feitos em Chromium/Edge; Firefox e Safari não foram executados (seção 9.2).
- **Sem novas dependências:** nada de bibliotecas — tudo é JavaScript e CSS puros, na mesma linha do restante do projeto.

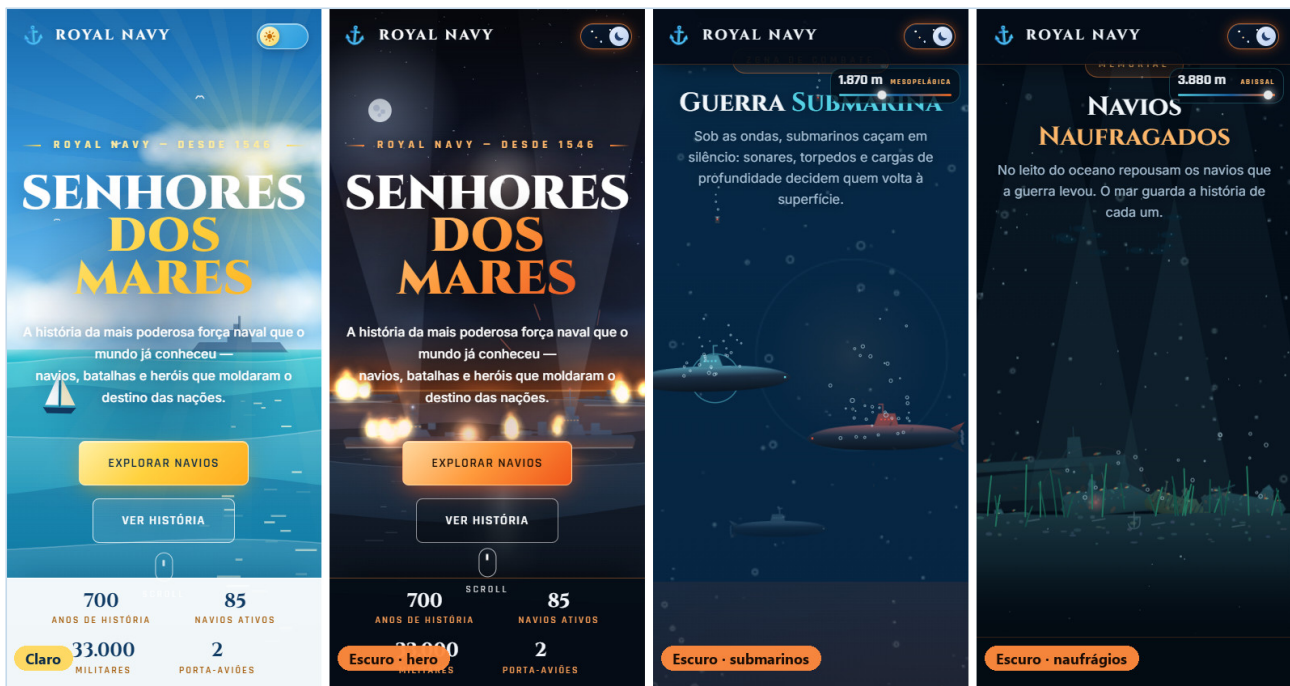


Figura 16 — No celular (390×844, emulação): modo claro e três telas do escuro; o botão fica na extremidade direita da barra, sem o rótulo.

6. Diagnóstico de desempenho

A causa da lentidão não era um único defeito, e sim a **soma de trabalho contínuo e invisível**: enquanto o usuário olhava o hero, o navegador também renderizava 12 cenas 3D fora da tela, repintava um grid animado, borrava fundos sobre canvases em movimento e recalculava layout a cada evento de rolagem. Cada item abaixo foi identificado por leitura do código; os mais graves foram confirmados por medição no navegador.

6.1 Problemas de maior impacto

#	Problema encontrado	Consequência	Solução aplicada
P1	13 contextos WebGL criados no carregamento, cada um com seu <i>loop</i> a 60 fps, mesmo fora da tela	GPU/CPU ocupadas o tempo todo; pico de travamento no <i>load</i> (13 renderers e compilações de shader); risco de perder contextos (limite $\approx$ 16)	Criação sob demanda + um loop único que só desenha o que está visível
P2	Verificação de <i>resize</i> comparava <i>canvas.width</i> (que já inclui o DPR) com <i>clientWidth</i>	Em telas com escala $\neq$ 100% (ex.: 125%), <i>setSize</i> rodava em todo frame , realocando o buffer de desenho	<i>ResizeObserver</i> : ajusta só quando o tamanho realmente muda
P3	Modal: o loop continuava após fechar; a cada abertura novos listeners em <i>window</i> e um novo renderer eram criados	Trabalho invisível permanente que crescia a cada uso; vazamento de memória	Renderer criado uma vez; loop para ao fechar; listeners por Pointer Events no próprio canvas
P4	12 mapas de sombra (1024 ²) com PCF suave, antialias e <i>tone mapping</i> em todas as cenas	Cada cena pequena pagava por sombras que mal se vêem	Sombras só no modal e na imagem grande; <i>tone mapping</i> removido
P5	Materiais e geometrias novos para cada peça; 26 esferas de luz + 16 portinholas + canhões como malhas separadas	Centenas de draw calls e de objetos por cena	Cache compartilhado + <i>InstancedMesh</i>
P6	<i>backdrop-filter</i> (blur 4–20 px) na navbar, na barra de estatísticas, nos selos 3D e no overlay do modal	O blur é reprocessado a cada mudança do que está atrás — e atrás havia canvases animando	Removido; fundos quase opacos
P7	<i>.hero::before</i> animava <i>background-position</i> em tela cheia, para sempre	Repintura da viewport inteira a cada frame (não é acelerável pelo compositor)	Substituído por raios de sol estáticos (uma única pintura)
P8	Brilho do cursor: loop infinito, alterando <i>left/top</i> e com <i>mix-blend-mode: screen</i> sobre tudo	Layout a cada frame + mistura de cores da página inteira	<i>translate3d</i> , sem <i>blend-mode</i> , loop para ao convergir, só com mouse

6.2 Problemas de impacto médio

#	Problema	Solução
P9	Hero 2D: 6 createLinearGradient por frame, 120 estrelas, partículas, sem pausa fora da tela; tempo por frame ($t += 0.016$)	Gradientes/sprites pré-renderizados; pausa por visibilidade; tempo em segundos (dt)
P10	Handler de scroll lia offsetTop de todas as seções (layout forçado) e escrevia estilos a cada evento	Um handler com rAF que só lê scrollY; link ativo por IntersectionObserver
P11	transition: all em ~20 componentes	Listas explícitas de propriedades (transform, box-shadow, cores...)
P12	Barras da frota animavam width (layout)	transform: scaleX
P13	Contadores com setInterval(16 ms)	rAF com easing por tempo
P14	Animação infinita de escala em uma camada de seção inteira (batalhas), quase invisível	Removida
P15	Scripts sem defer e sem verificação de integridade; 12 pesos de fonte; sem preconnect para o cdnjs	defer + SRI (sha512), 9 pesos, preconnect
P16	Canvases WebGL com alpha:true; oceano transparente; 4 luzes por cena	Canvas opaco; oceano opaco; 2 luzes
P17	Sem prefers-reduced-motion	Suporte completo (CSS e JS)

6.3 Exemplos de código (antes → depois)

P2 — resize a cada frame:

```
// ANTES (todo frame): em DPR 1,25 a condição é SEMPRE verdadeira
if (renderer.domElement.width !== cw || renderer.domElement.height !== ch) {
  renderer.setSize(cw, ch, false); // realoca o buffer de desenho
}
// DEPOIS: ResizeObserver marca "resizePending" só quando o tamanho muda
const ro = new ResizeObserver(list => list.forEach(r => byCanvas.get(r.target).resizePending = true));
```

P7 — grid animado do hero:

```
/* ANTES: repinta a tela inteira a cada frame, para sempre */
.hero::before { background-size: 60px 60px; animation: gridDrift 20s linear infinite; }
@keyframes gridDrift { to { background-position: 60px 60px; } }

/* DEPOIS: raios de sol estáticos, uma única pintura */
.hero::before { background: repeating-conic-gradient(from 195deg at 78% 24%, ...); }
```

7. Erros e bugs corrigidos

Esta seção separa **bugs funcionais** (comportamento errado) dos problemas de desempenho da seção anterior. Os itens marcados com **[R]** foram **reproduzidos no navegador** antes de serem corrigidos; os demais foram identificados por leitura do código e verificados depois da correção.

7.1 Interface e animações

#	Bug	Causa raiz	Correção
B1 [R]	O "levantar" dos cards no hover nunca acontecia (ships, batalhas, linha do tempo, frota)	.fade-in-up.visible e .ship-card: hover têm especificidade (0,2,0) e a primeira vem depois no arquivo, então translateY(0) vencia	Classes de revelação são removidas no fim da transição; o elemento volta ao CSS normal. Hover verificado: matrix(1,0,0,1,0,-8)
B2 [R]	Hover lento (0,7 s) e sem transição de sombra/borda	.fade-in-up definia transition: opacity 0.7s, transform 0.7s e sobrescrevia a do componente	Mesma correção de B1
B3 [R]	Barras da seção Frota ficavam em 0 px para sempre	O JS zerava o style inline <i>antes</i> de ler a largura original; a regex depois só encontrava "0px"	Largura passa a vir de --fill no HTML; o JS só adiciona uma classe; a barra anima por scaleX
B4	Parallax do título não funcionava	A animação CSS com forwards tem precedência sobre estilos inline e travava transform/opacity	Animação removida do contêiner (os filhos animam); parallax via rAF
B5	Erro no console ao clicar nos links do rodapé ("'#' is not a valid selector")	Links href="#" passavam pelo handler de scroll suave, que fazia querySelector('#')	Links usam data-open-ship; scroll suave passou para o CSS
B6	Atributos data-aos sem biblioteca AOS	Código morto (7 elementos)	Removidos
B7	Cards filtrados ainda recebiam foco por teclado	Só se usava pointer-events: none	Atributo inert + aria-pressed nos botões
B8	Em telas baixas o conteúdo do hero passava por trás da barra de estatísticas (botões escondidos)	Barra absoluta sem reserva de espaço	Padding reservado no hero; título limitado por 17vh
B9	Card de destaque (Trafalgar) com título e selo empurrados para o canto	Grid com coluna <i>auto</i> e itens sem posição explícita: o texto longo alargava a coluna	Posições explícitas de coluna/linha; versão de coluna única no celular

7.2 Modal

#	Bug	Causa raiz	Correção
B10	Conteúdo cortado: Armamento e História não apareciam e não rolavam	Linha do grid <i>auto</i> crescia com o conteúdo além do <code>max-height: 90vh</code> ; <code>overflow: hidden</code> cortava	<code>grid-template-rows: minmax(0, 1fr)</code> : o painel direito rola (1192 px em 691 px, medido)
B11	Navio deslocado para baixo e fora do quadro	O canvas seguia a altura inflada da linha do grid; câmera fixa para um quadro largo	Correção do grid + câmera com enquadramento automático
B12	Sem gerenciamento de foco, ESC ou armadilha de Tab; rolagem do fundo saltava ao abrir	Ausentes	Foco no botão fechar, Tab preso, Esc fecha, foco devolvido, <code>scrollbar-gutter: stable</code>
B13	Renderização continuava após fechar; memória crescia a cada abertura	Loop nunca cancelado; <code>render</code> e <code>listeners</code> recriados	Loop pausado ao fechar; <code>render</code> único
B14	Botão fechar com <code>onClick</code> inline e handlers globais	Estilo do código original	Delegação de eventos (<code>data-open-ship</code> , <code>.modal-close</code>)

7.3 Servidor (app.py)

#	Bug	Causa raiz	Correção
B15 [R]	O servidor não iniciava quando a saída não era um console (<code>UnicodeEncodeError</code>)	Banner com caracteres Unicode (traço duplo e emojis de âncora e globo) em cp1252 (Windows)	Banner só ASCII. Verificado: sobe sem <code>-X utf8</code>
B16	Risco de segurança: <code>debug=True</code> com <code>host="0.0.0.0"</code>	O debugger do Werkzeug executa código Python; ficava acessível para toda a rede local	Host padrão 127.0.0.1; debug só é permitido em host local (desligado automaticamente caso contrário)
B17	Rota <code>/static/<path></code> duplicada	O Flask já registra a rota estática automaticamente	Rota removida
B18 [R]	Arquivos inexistentes (ex.: <code>/favicon.ico</code>) recebiam a página inteira (≈ 25 KB) com status 404	O handler 404 sempre renderizava o <code>index</code>	404 real de 9 bytes para <code>/static/*</code> e URLs com extensão; <code>favicon</code> embutido evita a requisição
B19	Sem cache para arquivos estáticos	Configuração padrão	1 h fora do debug; sem cache em debug

7.4 Erros de julgamento meus, corrigidos durante o trabalho

Registro honesto de ajustes feitos depois da primeira versão, encontrados nas capturas de tela:

- Mar 3D “estourado” em ciano neon: a soma das luzes passava de 1,0. Reduzidas as intensidades e escurecida a água.
- Velas do galeão esverdeadas: o hemisfério refletia o mar ciano nas superfícies verticais. Cor de solo neutralizada.
- Ícone das espadas cruzadas invisível nas Batalhas: glifo de texto herdava cor escura sobre fundo azul. Definida cor clara.
- Barra de estatísticas abaixo da dobra em 1366×768: padding do hero reduzido.
- Comentário/commit dizia “12 → 8 fontes”; o correto é 12 → 9 pesos. Corrigido antes do envio.

7.5 Ajustes e correções durante o desenvolvimento do modo escuro (v2.0)

Achados nas capturas de tela e nas medições, corrigidos antes da publicação:

#	Achado	Causa	Correção
D1 [R]	A batalha do hero ficava pouco visível	Cascos escuros demais, quase da cor do fundo, e a vinheta CSS do .hero-overlay escurecia o centro da tela	Cascos mais claros (#1b2d45 / #243a56) com luar por cima e brilho de incêndio por baixo; faixa quente refletida no mar; navios, explosões e fogo maiores; mais tiros por segundo; vinheta central reduzida
D2 [R]	O medidor de profundidade cobria o conteúdo	Primeira versão: coluna vertical sobre as seções	HUD horizontal fixo logo abaixo da navbar (top: 82px; 74 px no celular), sem capturar cliques
D3	Emenda visível entre Guerra Submarina e Batalhas	O fundo da seção Batalhas começava com um brilho opaco	Degradê que começa transparente (0 → 42% aos 220 px) e brilhos radiais mais discretos (0,22 → 0,12); depois, uma máscara de 320 px no topo de .battles-bg eliminou a borda que ainda restava
D8	Zonas do medidor com limites diferentes dos oceanográficos (2.180 m aparecia como "Mesopelágica")	Limites simplificados (150 / 900 / 2.600 / 3.700 m)	Limites usuais: epipelágica 0–200 m, mesopelágica 200–1.000 m, batipelágica 1.000–4.000 m, abissal 4.000 m+ (ZONES em deep-sea.js)
D4	No celular os navios da batalha ficavam pequenos	Tamanhos proporcionais à largura (390 px)	Fator ×1,34 quando a largura é menor que 600 px
D5	O modo claro pagava por trabalho do modo escuro	O fundo do mar media o layout a cada redimensionamento mesmo com o tema claro	remeasure() sai sem medir enquanto o tema é claro; deep-canvas, medidor e faixas com display:none no claro
D6	Medição de FPS travava	O painel do navegador do aplicativo, quando oculto, não dispara requestAnimationFrame	Medições passaram a usar o Edge headless via DevTools Protocol (primeiro por software; depois com a GPU — ver seção 9)
D7	Comparação de pixels com um artefato (barra de navegação duplicada)	Comparava capturas de sessões diferentes; o Edge às vezes captura elementos fixos duplicados	Comparação refeita com a v1.1 publicada (via git) e a v2.0 servidas nas mesmas condições: 0 pixel de diferença no corpo

Dica de operação descoberta no processo: ao encerrar o Flask em modo debug, o recarregador deixa um segundo processo Python segurando a porta 5000; é preciso encerrar os dois.

8. Paletas: dia ensolarado e noite de guerra

A identidade anterior era noturna (navy profundo, aço e ouro escuro, estrelas). A nova é **diurna**: o que se vê num dia de sol em alto-mar — céu azul, água turquesa, luz dourada, espuma branca e o vermelho-coral das boias. A mudança é global e vive em **variáveis CSS** (:root), o que permite trocar o tema editando um único bloco.

Céu

sky-50 #F3FBFF	sky-100 #E3F4FD	sky-200 #C4E6F9	sky-300 #92D2F3	sky-400 #57B8EA	sky-500 #2A9BDC	sky-600 #1A7DBB
-------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------

Mar turquesa

sea-300 #6FE3D8	sea-400 #2EC9C2	sea-500 #0FA9B1	sea-600 #0A8795	sea-700 #076A78
--------------------	--------------------	--------------------	--------------------	--------------------

Sol

sun-100 #FFF7D9	sun-200 #FFEC9F	sun-300 #FFDB5C	sun-400 #FFC62E	sun-500 #F9A71A	sun-600 #E58A0B	sun-700 #B96500	sun-800 #A35700
--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------

Azul-marinho (texto e rodapé)

navy-900 #0A2A4A	navy-800 #0F3A63	navy-700 #164A7C	navy-600 #1F5D96
---------------------	---------------------	---------------------	---------------------

Coral e areia

coral-400 #FF8060	coral-500 #F2593A	sand-50 #FFFDF6	sand-100 #FDF3DC
----------------------	----------------------	--------------------	---------------------

Onde cada cor é usada

Papel	Cores	Aplicação
Texto principal / títulos	navy-900, navy-800	Títulos de seção, nomes de navios, números
Texto secundário	#3b5673 (7,7:1 sobre branco)	Parágrafos e legendas
Destaques de seção	sea-600 / sea-700	Palavra em destaque nos títulos, classes dos navios, botão “Ver detalhes”
Ação e ênfase	sun-200 ... sun-500	Botão principal, selos, anos da linha do tempo, resultado das batalhas, sublinhado do menu
Rótulos pequenos sobre claro	sun-800, sea-700	Escolhidos para atingir ≥ 4,5:1 em textos pequenos
Interação/alerta	coral-400/500	Ponto da linha do tempo no hover; botão fechar no hover
Superfícies	branco, sky-50/100, sand-50	Cards e fundos de seção, alternando frio e quente
Seção de contraste	gradiente #0e4d86 → #0a6aa0 → #0a8a9c	Batalhas: mar profundo iluminado por um brilho dourado no canto
Rodapé	navy-900	Âncora visual final; separado por uma onda

Elementos de identidade novos

- **Hero diurno:** sol com halo e raios, nuvens à deriva, gaivotas, veleiro, silhueta de um navio de guerra ao horizonte, ondas com crista brilhante e cintilação do sol na água.
- **Cenas 3D ensolaradas:** céu em gradiente, mar turquesa, luz quente do sol, hulls em cinza-naval claro, velas cor de creme.
- **Ondas divisórias:** máscara SVG reutilizável (colorida por background) entre Navios → Batalhas → Frota e antes do rodapé.
- **Vinheta de contraste no hero:** garante leitura do texto branco sobre o céu e o mar.

Acessibilidade de cor

Texto secundário e rótulos pequenos usam tons escuros (sun-800, sea-700, #3b5673) para atingir contraste mínimo de 4,5:1 sobre fundo claro. Sobre o hero (imagem viva), a leitura é apoiada por vinheta escura suave e *text-shadow*. O contraste do texto grande do hero é adequado, mas o parágrafo sobre o mar tem contraste mais modesto; ficou registrado como ponto de atenção na seção 10.



Figura 17 — Batalhas: seção de contraste em azul profundo, card de destaque corrigido e ícone claro.

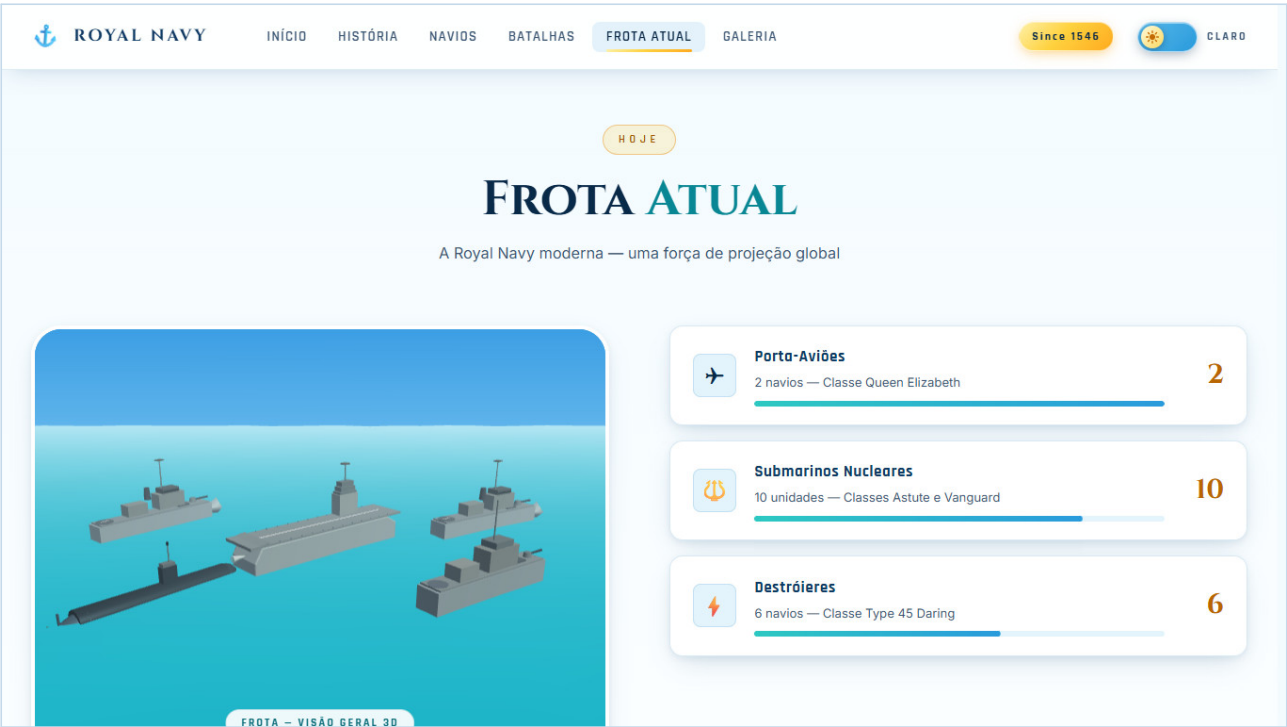


Figura 18 — Frota Atual: cena 3D em formação e barras animadas até seus valores (2, 10 e 6 unidades).

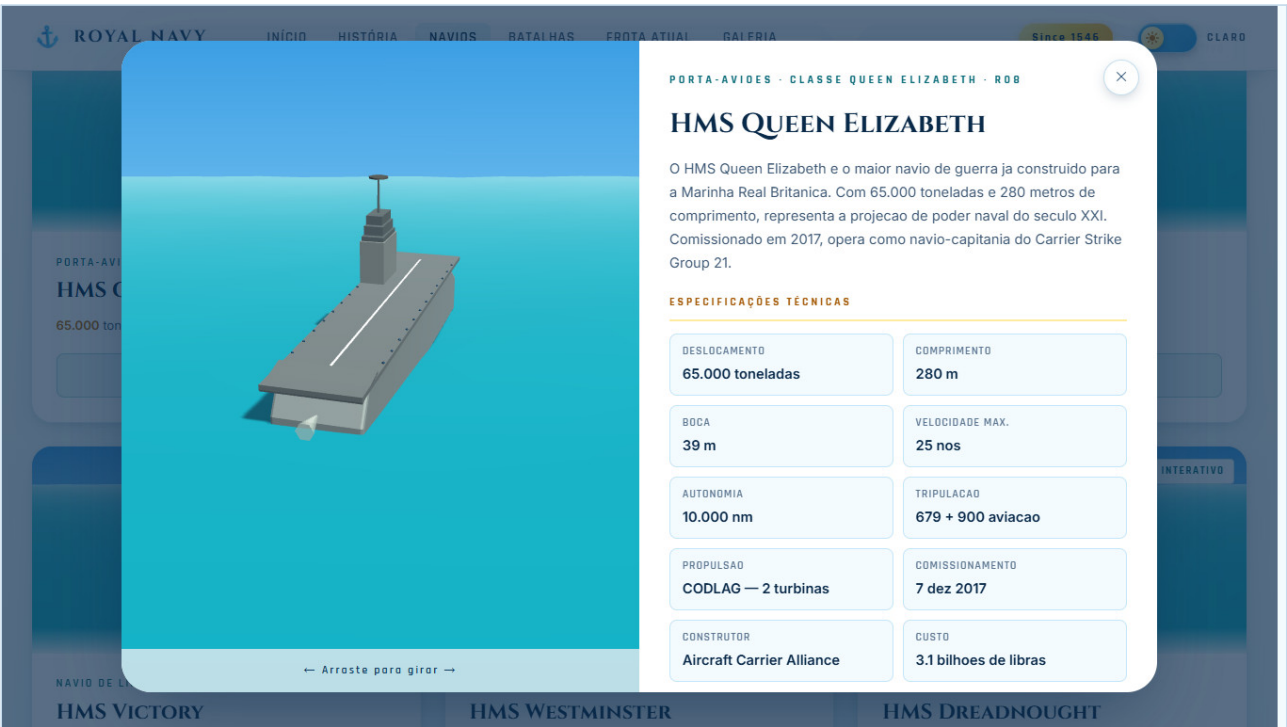


Figura 19 — Modal do HMS Queen Elizabeth: navio enquadrado, ficha técnica e botão fechar.

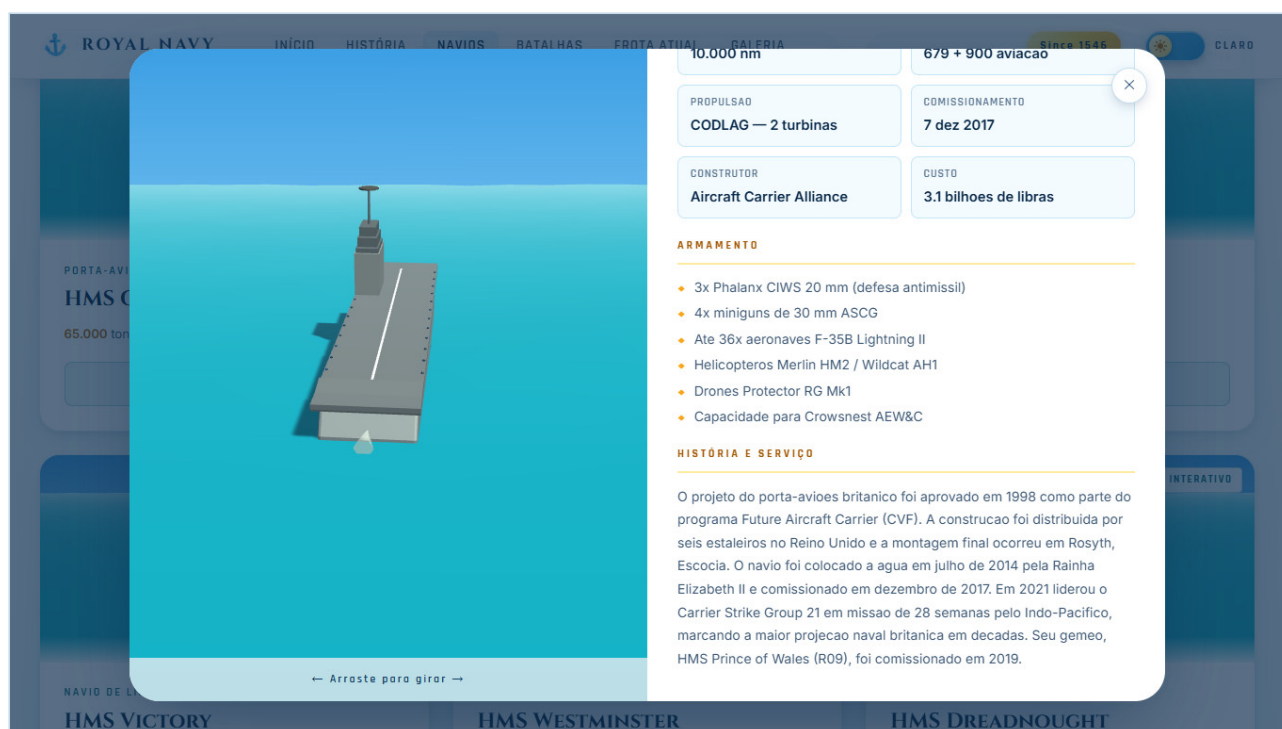


Figura 20 — O painel do modal rolado: Armamento e História agora acessíveis.

Paleta do modo escuro: noite de guerra

O modo escuro troca o dia por uma noite de tempestade: **noite e aço** no fundo, **brasa** (laranja) para o que é ação e ênfase — fogo, títulos, botões, anos — e **ciano** para o que é água e informação. As cores vivem em variáveis CSS no bloco `:root[data-theme="dark"]` de `dark.css` e, para as cenas, em constantes dos arquivos JavaScript.

Noite e aço

night #050B14	body #04101C	steel #0A1A2E	sky-mid #0B1C33	hull #1B2D45	super #243A56
-------------------------	------------------------	-------------------------	---------------------------	------------------------	-------------------------

Brasa

ember-l #FFB86B	ember #FF8A3D	fire #FF6A1F	fire-d #E8431B	glow #FFE2A8
---------------------------	-------------------------	------------------------	--------------------------	------------------------

Ciano e água (do raso ao fundo)

cyan #54D1E4	cyan-l #7FDCEC	d 0 #0E6276	d 0,12 #0A4C68	d 0,30 #083454	d 0,55 #051F3A	d 1 #030C14
------------------------	--------------------------	-----------------------	--------------------------	--------------------------	--------------------------	-----------------------

Textos

tx #EEF5FB	tx2 #A9C1D6	tx3 #8FA9C0
----------------------	-----------------------	-----------------------

Contraste (WCAG) no modo escuro

Texto	Sobre o fundo do corpo (#04101c)	Sobre o painel translúcido no pior caso (água rasa)
--d-tx #eef5fb (texto principal)	17,4 : 1	14,3 : 1
--d-tx2 #a9c1d6 (secundário)	10,3 : 1	8,5 : 1
--d-tx3 #8fa9c0 (rótulos)	7,9 : 1	6,5 : 1
ember-l #ffb86b (anos, destaques)	11,2 : 1	9,3 : 1
ember #ff8a3d	8,2 : 1	6,7 : 1
cyan-l #7fdcec (classes dos navios)	12,2 : 1	10,0 : 1
Botão principal: #1a0d05 sobre #ffd9a0 → #ff9a3c → #f0561b	14,2 : 1 → 9,0 : 1 → 5,5 : 1	—

Todos os pares ficam acima de 4,5:1 (nível AA). O pior caso do painel considera o painel rgba(6,22,40,0.8) sobre a água mais clara (#0e6276).

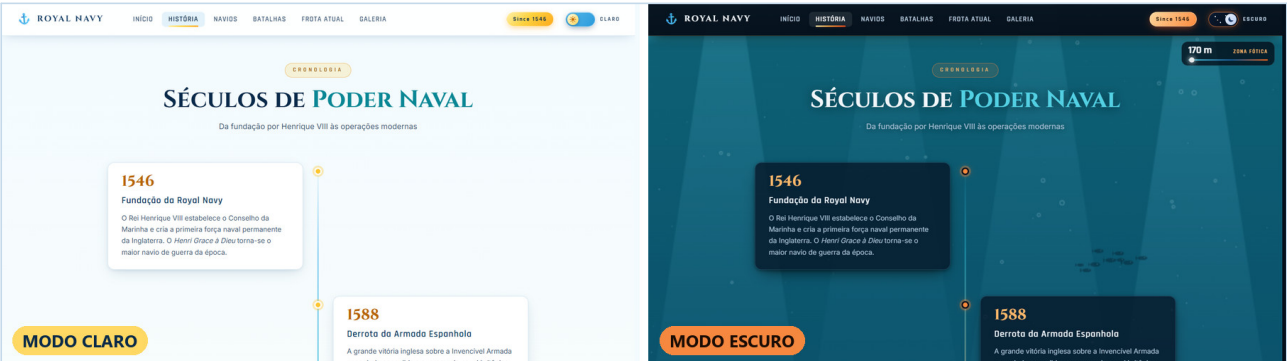


Figura 21 — Seção História nos dois modos: mesmo conteúdo e layout; no escuro, a água do canvas fixo aparece atrás dos cards translúcidos.

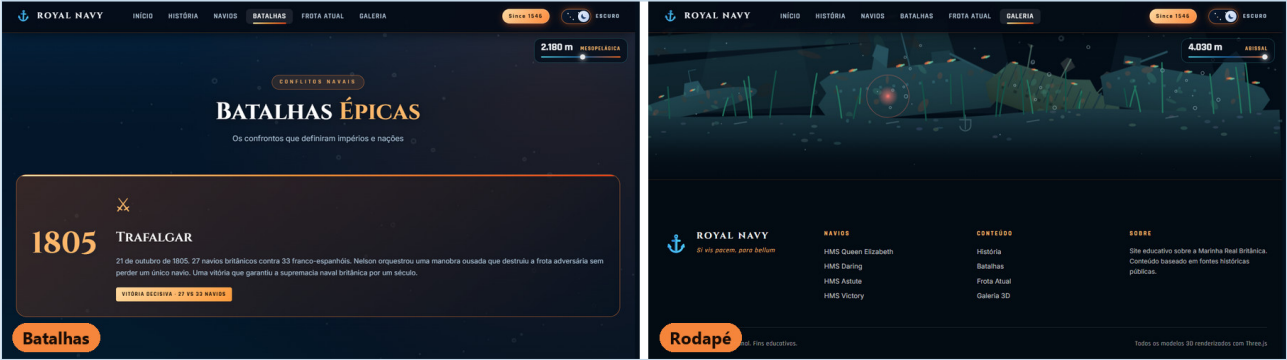


Figura 22 — Modo escuro: Batalhas (sem as ondas claras) e rodapé (leito do mar).

9. Como foi verificado

9.1 Métodos

Método	Ferramenta	O que confirmou
Leitura completa do código	Revisão manual (6 arquivos)	Lista de problemas P1–P17 e B1–B19
Medição de FPS	Navegador do aplicativo (rAF durante 3 s)	1,7 FPS (original) → 56,7 (topo) / 60 (navios visíveis)
Inspeção do CSS calculado	<code>document.styleSheets + getComputedStyle</code>	Conflito <code>.fade-in-up × :hover</code> (B1/B2); barras em 0 px (B3)
Contagem de draw calls	Interceptação de <code>drawElements/drawArrays</code> em WebGL/WebGL2	≈ 77/frame com navios; ≈ 17/frame só o modal; 0 no topo
Capturas por seção	Edge headless via Chrome DevTools Protocol (script próprio)	Revisão visual de todas as seções em 1366×768 e 390×844
Hover real	Evento de mouse sintético do DevTools	Transform do card = <code>translateY(-8 px)</code>
Rolagem do modal	Medição de <code>scrollHeight × clientHeight</code> (desktop)	1192 px de conteúdo em 691 px visíveis; <code>overflow-y: auto</code>
Console	Captura de erros e avisos em cada execução	Nenhum erro ou aviso
Rotas do servidor	<code>curl</code>	<code>/</code> = 200; <code>/static/*</code> = 200; arquivo inexistente = 404 (9 bytes); URL desconhecida = 404 + página
Sintaxe	<code>node --check, py_compile</code>	Todos os arquivos válidos
Versão estática num subcaminho	<code>http.server</code> + Edge headless	Site idêntico ao do Flask, sem erros
Política CSP com teste de controle	Domínio <i>Log</i> do DevTools	Política real: 0 violações. Política propositalmente errada: bloqueio detectado (o teste enxerga violações)
Site publicado	<code>curl</code> no endereço do Pages	<code>index.html</code> e os arquivos de static respondendo HTTP 200 (20/09/2026)
Disponibilidade do domínio	RDAP do Registro.br + <code>nslookup</code>	<code>royalnavy.com.br</code> não registrado (404 no RDAP; controle <code>google.com.br</code> = 200; DNS inexistente)
Trava do domínio	Execução de <code>enable_custom_domain.py</code>	Recusou domínio inexistente e domínio de outro servidor; nada foi gravado
v2.0 · Comparação de pixels	Servir a v1.1 (git) e a v2.0 e comparar capturas com PIL	Modo claro: 0 pixel de diferença no corpo (seção 9.6)
v2.0 · Desempenho por estado e por rolagem	Edge headless com GPU; contagem de quadros do rAF; medianas de rodadas	Seção 9.3
v2.0 · Canvases ativos	Instrumentação das chamadas de desenho por canvas	Só o que está visível desenha (seção 9.4)
v2.0 · Sistema de temas	Cliques, recarga, API desativada, sondagem da animação	Seção 9.5
v2.0 · Quadros da revelação	Animação congelada em 4 pontos (Web Animations API)	Figura 10

Método	Ferramenta	O que confirmou
v2.0 · Site publicado	curl + Edge headless no endereço do Pages	Seção 9.7

9.2 Limitações honestas

- As medições de FPS foram feitas em **WebGL por software**; não foram feitas na sua máquina com GPU real.
- Não houve teste em navegadores diferentes do Chromium/Edge (Firefox e Safari não foram executados).
- Não há testes automatizados no projeto; a verificação foi por medição, captura e inspeção.
- O celular foi verificado por emulação (390×844), não em dispositivo físico; toque e gestos não foram exercitados em hardware real.
- O código original não estava em um repositório git, então não há histórico anterior; as versões antigas podem ser recuperadas pelo histórico do OneDrive.
- v2.0:** as medições de FPS foram feitas em um único computador (gráficos integrados Intel UHD); as de rolagem têm apenas 2 rodadas, e a variação entre rodadas é de até $\pm 10\%$.
- v2.0:** o modo escuro foi testado em Chromium/Edge (desktop e emulação de celular 390×844); Firefox e Safari não foram executados, nem aparelhos móveis reais.

9.3 Desempenho medido na v2.0

Máquina: Intel Core i3-1215U, gráficos integrados Intel UHD, 7,7 GB de RAM; Edge com GPU (D3D11), janela 1366×768, DPR 1; monitor a 120 Hz (uma página sem canvas desenhando chega a ≈ 120 fps). Cada valor é a **mediana de 3 rodadas** contando quadros do requestAnimationFrame por 3 s. A v1.1 foi servida a partir do commit d006295, nas mesmas condições.

Estado (fps · ms por quadro)	v1.1 claro	v2.0 claro	v2.0 escuro
Topo: hero animado	62 · 16,2	61 · 16,3	61 · 16,3
Grade de navios (6 cenas WebGL)	93 · 10,8	82 · 12,2	61 · 16,5
Galeria (5 cenas WebGL)	85 · 11,8	73 · 13,7	58 · 17,2
Seção sem canvas (Batalhas) / só mar profundo (História)	120 · 8,3	120 · 8,3	120 · 8,3
Faixa Guerra Submarina	—	—	120 · 8,3
Faixa Navios Naufragados	—	—	90 · 11,1

Rolagem contínua do topo ao fim da página em 18 s (mediana de 2 rodadas):

Métrica	v1.1 claro	v2.0 claro	v2.0 escuro
Altura da página	8.941 px	8.941 px	10.294 px (mais duas faixas)
FPS médio	81,1	78,1	68,6
Tempo de quadro: mediana · p95 · p99 (ms)	8,4 · 24,9 · 83	8,4 · 25,2 · 79	8,4 · 29,1 · 104
Quadros acima de 33 ms · acima de 50 ms	36 · 20	36 · 20	40 · 22
Nível do RNPerf ao final	—	— (desligado)	0 nas duas rodadas (o monitor reduziu os efeitos)

Leitura: no modo claro a v2.0 fica na mesma faixa da v1.1 — a variação entre rodadas nas cenas 3D chega a $\pm 10\%$ nesta máquina. No modo escuro o hero em batalha custa o mesmo que o hero claro; trechos com só o mar profundo e a guerra

submarina chegam ao teto do monitor; nas seções com 3D o modo escuro fica ≈ 60 fps (o fundo do mar é desenhado junto com as cenas WebGL). Na rolagem contínua o escuro tem $\approx 12\%$ menos FPS médio e o RNPerf reduziu a qualidade ao mínimo; a mediana de quadro é a mesma nos três (8,4 ms).

Uma medição anterior, com o Edge renderizando por *software* (SwiftShader), deu ≈ 6 fps já no hero **claro original**; por isso os valores absolutos daquela configuração não foram usados.

9.4 Quais canvases desenhavam em cada trecho

Contagem de chamadas de desenho por canvas em cada posição da página (canvases com mais de 1,5 chamada/s):

Modo · posição	Canvases 2D desenhando	WebGL
Claro · topo	ocean-canvas (hero)	—
Claro · grade de navios / galeria	nenhum	sim (cenas visíveis)
Claro · Batalhas	nenhum (após voltar do escuro: nenhum também)	—
Escuro · topo	ocean-canvas (batalha)	—
Escuro · rolado 400 px	ocean-canvas + deep-canvas	—
Escuro · História	deep-canvas	—
Escuro · grade de navios	deep-canvas	sim
Escuro · Guerra Submarina	deep-canvas + sub-canvas	—
Escuro · naufrágios / rodapé	deep-canvas + wreck-canvas	—

9.5 Sistema de temas: testes

Teste	Resultado
Persistência	Tema escuro salvo em localStorage; após recarregar: data-theme=dark, theme-color #050b14, aria-checked=true, rótulo “Escuro”, fundo do mar visível
6 cliques rápidos (120 ms entre eles)	Só o primeiro foi aceito; estado final consistente (tema, aria-checked e armazenamento iguais) e a classe theme-switching removida ao fim
Cortina de reserva (API desativada)	Cortina presente durante a troca, removida depois; tema aplicado; sem erros
Parâmetros da revelação	Animação criada em ::view-transition-new(root) com 900 ms e cubic-bezier(.4,0,.2,1); termina no estado finished e limpa theme-switching
Console e CSP	Nenhum erro ou violação em todos os testes

9.6 Modo claro sem regressões

A v1.1 publicada (commit d006295) e a v2.0 foram servidas nas mesmas condições e as seções História, Linha do tempo e Batalhas foram comparadas pixel a pixel:

Seção	Pixels diferentes no corpo (abaixo dos 70 px da navbar)	Pixels diferentes na navbar
História	0	14.375
Linha do tempo	0	14.590
Batalhas	0	14.602



Figura 23 — Batalhas (modo claro): v1.1, v2.0 e a diferença (em vermelho). A única diferença está na barra de navegação, que ganhou o botão de tema.

Além disso, o `style.css` teve 116 linhas acrescentadas e 0 alteradas (`git diff --stat` entre os commits).

9.7 Verificação no site publicado

- Commit `ae6b9ea` e tag `v2.0.0` enviados; remoto igual ao local.
- O GitHub Pages atualizou em ≈ 15 s; `theme-init.js`, `theme.js`, `war-hero.js`, `deep-sea.js` e `dark.css` responderam HTTP 200 e o HTML publicado contém o botão.
- No endereço público, o modo escuro foi ligado e percorrido no Edge headless: sem erros de console nem violações de CSP.

10. Pendências e recomendações

Prioridade	Item	Detalhe
Alta	Acentuação em <code>ships-data.js</code>	O arquivo de dados está sem acentos (“Porta-Aviões”, “Britânica”, “projecao”), diferente do restante do site. Não foi alterado por estar fora do escopo pedido; é uma correção simples e de alto valor.
Alta	Menu para celular	Abaixo de 768 px os links do menu somem e não há alternativa (hambúrguer). A navegação por âncoras fica indisponível.
Alta	Registrar <code>royalnavy.com.br</code>	Concluir a compra no Registro.br (só o titular pode) e criar os registros DNS; depois rodar <code>enable_custom_domain.py</code> . Passo a passo na seção 12. O domínio estava disponível em 20/09/2026.
Concluída	Publicar como site	Feito: versão estática em <code>docs/</code> servida pelo GitHub Pages em HTTPS (seção 12).
Média	Contraste do parágrafo do hero	O texto branco médio sobre o mar tem contraste modesto; considerar aumentar a vinheta ou o peso da fonte.
Média	Hospedar <code>three.js</code> localmente	Hoje vem do cdnjs (com SRI). Hospedar em <code>static/</code> elimina uma dependência externa e funciona offline (≈ 600 KB).
Média	Compressão <code>gzip/brotli</code>	O servidor de desenvolvimento não comprime; em produção use um servidor/host que comprima (o GitHub Pages já faz).
Baixa	Limite de contextos WebGL	O site usa até 13 contextos após rolar tudo (limite ≈ 16). Uma evolução seria um único renderer compartilhado, ou liberar cenas muito distantes.
Baixa	Testes automatizados	Um pequeno teste de fumaça (Playwright) para carregar a página, checar o console e abrir o modal evitaria regressões.
Baixa	Quebra de linha do texto do hero	O <code>
</code> fixo gera linhas curtas no celular.

11. Operação: executar, versionar e publicar

11.1 Executar localmente

```
python -m venv .venv
.venv\Scripts\activate          # Windows (Linux/macOS: source .venv/bin/activate)
pip install -r requirements.txt
python app.py                   # abre em http://localhost:5000
```

11.2 Variáveis de ambiente

Variável	Padrão	Função
PORT	5000	Porta do servidor
HOST	127.0.0.1	Interface de rede. 0.0.0.0 expõe na rede local; nesse caso o modo debug é desligado automaticamente
FLASK_DEBUG	1	0 desliga o recarregamento automático e ativa cache de 1 h nos estáticos

11.3 Repositório

Item	Valor
URL	https://github.com/AlanSouzaDev7/Naval_Design-WEB-V.1
Branch	main (rastrea origin/main)
Site publicado	https://alansouzadev7.github.io/Naval_Design-WEB-V.1/ (Pages: branch main, pasta /docs)
Commits	a450870 — otimização e paleta · f26ebc1 — versão estática e segurança · mais o commit desta documentação e do preparo do domínio (ver <code>git log</code>)
Arquivos versionados	código, README, .gitignore, requirements, scripts de build/domínio, docs/ e este PDF
Fora do repositório	.venv, .claude/, arquivo teste, caches Python

11.4 Guia rápido de manutenção

Quero...	Edite...
Trocar as cores do site	Bloco <code>:root</code> no início de <code>style.css</code> (e as cores de C em <code>ships-3d.js</code> para as cenas 3D; e os gradientes do hero em <code>initHeroSea</code>)
Adicionar um navio	1) Entrada em <code>ships-data.js</code> ; 2) card em <code>index.html</code> com <code>data-ship</code> e <code>data-type</code> ; 3) se for um tipo novo, uma função <code>buildX()</code> e um case em <code>buildShip</code>
Ajustar o brilho/luz 3D	<code>createScene()</code> em <code>ships-3d.js</code> (<code>HemisphereLight</code> e <code>DirectionalLight</code>)
Mudar textos e seções	<code>templates/index.html</code>
Mudar cache/porta/segurança	<code>app.py</code> (Flask local) e a meta CSP em <code>index.html</code> (vale também no Pages)
Atualizar o site publicado	Edite <code>templates/static</code> , rode <code>python build_static.py</code> , faça commit e push (o Pages atualiza sozinho em ~1 min)
Ativar o domínio próprio	<code>python enable_custom_domain.py royalnavy.com.br</code> — só depois do registro e do DNS
Adicionar animação de entrada	Inclua o seletor na lista <code>groups</code> de <code>initReveal()</code> (<code>main.js</code>)

11.5 Histórico de versões

Versão	Data	Commit(s)	O que mudou
1.0 (original)	—	sem git	Site recebido: Flask + HTML/CSS/JS + Three.js, identidade noturna, 1,7 FPS
1.1	19–20/09/2026	a450870, f26ebc1, d006295	Auditoria, desempenho, correção de bugs, paleta ensolarada, versão estática (docs/), CSP e SRI, preparo do domínio, PDF v1.1
2.0.0	26/09/2026	ae6b9ea · tag v2.0.0	Modo escuro “Guerra em alto mar”: botão e transição, hero em batalha, mergulho, guerra submarina, naufrágios, 3D noturno, RNPerf; 19 arquivos (+4.918 / –177 linhas, incluindo docs/)
2.0.1	26/09/2026	v2.0.1	Zonas do medidor de profundidade com limites oceanográficos usuais; emenda visual entre Guerra Submarina e Batalhas eliminada (ver D3 e D8, seção 7.5)

11.6 Guia de manutenção do modo escuro

Quero...	Edite...
Trocar as cores do modo escuro	Tokens em <code>:root[data-theme="dark"]</code> (dark.css); <code>THEMES.dark</code> (ships-3d.js); cascos e mar em <code>war-hero.js</code> ; <code>STOPS</code> (cor da água) em <code>deep-sea.js</code>
Ajustar a intensidade da batalha	<code>buildShips()</code> em <code>war-hero.js</code> (rate = intervalo entre salvos, <code>dmgMax</code> = acertos para afundar); <code>CAP</code> e <code>POOL</code> para partículas
Mudar profundidade e zonas do medidor	<code>ZONES</code> e a escala 4200 em <code>Gauge</code> (deep-sea.js)
Criar outra faixa submarina	Um <code><div class="abyss-band"></code> com canvas em <code>index.html</code> e uma chamada <code>band('id', onResize, onFrame)</code> em <code>deep-sea.js</code>
Ajustar a qualidade adaptativa	Limites 26 ms, 18,5 ms e 14 s em <code>sample()</code> (theme.js)
Ajustar a animação de troca	<code>set()</code> e <code>veilTransition()</code> (theme.js): duração 900 ms, easing, cortina 520 + 380 ms
Publicar uma nova versão	<code>python build_static.py</code> , <code>commit</code> , <code>git tag -a vX.Y.Z</code> e <code>git push --tags</code>

12. Publicação, domínio e segurança

12.1 Por que o localhost:5000 não funcionava no GitHub

localhost significa “este computador”. O endereço `http://localhost:5000` só existe na máquina de quem executa `python app.py` e apenas enquanto o servidor estiver aberto. Quem abre esse link em outro computador (ou no celular) tenta falar com a *própria* máquina, onde não há nada rodando.

Afirmiação	Realidade
“O GitHub roda o meu site”	O GitHub armazena o código. Ele não executa Python nem o Flask.
“localhost é um endereço público”	É um endereço privado da máquina local (127.0.0.1); ninguém de fora consegue acessá-lo.
“Preciso de um servidor Flask ligado”	Não. O template não usa lógica Jinja e nenhuma rota devolve dados dinâmicos: o site é 100% estático.

Solução adotada: publicar o site como **estático** no GitHub Pages — gratuito para repositórios públicos, com HTTPS automático e sem nenhum servidor seu ligado.

12.2 Como a publicação funciona

```

templates/index.html + static/      (fonte de verdade, usada também pelo Flask local)
    |
    | python build_static.py        troca "../static/" por "static/" e copia os arquivos
    v
docs/                          versão estática pronta (index.html, static/, .nojekyll)
    |
    | git push -> GitHub Pages (Settings > Pages: branch main, pasta /docs)
    v
https://alansouzadev7.github.io/Naval_Design-WEB-V.1/      (HTTPS, atualiza em ~1 min)

```

- **Por que reescrever os caminhos:** no Flask a página fica em / e ../static/ resolve para /static/. No Pages o site vive em um subcaminho (/Naval_Design-WEB-V.1/), então caminhos relativos simples (static/...) são necessários.
- **.nojekyll:** desliga o processamento Jekyll do Pages, que ignoraria arquivos e pastas iniciados por "_".
- **Uma só fonte:** a pasta docs/ é *gerada*; edite sempre templates/ e static/ e rode o build.
- **Verificação:** a versão estática foi testada num subcaminho local antes do envio e, depois de ativada, o endereço público respondeu HTTP 200 para a página e para CSS e JS.

12.3 Segurança

O maior ganho de segurança vem da própria arquitetura: um site estático **não tem servidor, banco de dados, login, formulários nem credenciais** para atacar. Sobre isso foram aplicadas camadas adicionais:

Medida	Onde	O que protege
HTTPS (TLS) automático	GitHub Pages	Criptografa o tráfego e impede adulteração no caminho
Content-Security-Policy (CSP)	<meta> em index.html (vale no Pages e no Flask)	Limita de onde scripts, estilos, fontes e imagens podem vir; reduz o impacto de injeção de código (XSS)
SRI (sha512) no three.js	Tag <script>	O navegador recusa o script se o arquivo do CDN for alterado
Referrer-Policy	meta + cabeçalho Flask	Envia o mínimo de informação de origem a terceiros
X-Content-Type-Options, X-Frame-Options, Permissions-Policy	Cabeçalhos do Flask (uso local)	Sem "adivinhação" de tipos, sem iframes e sem câmera/microfone/localização
Debug só em 127.0.0.1	app.py	O debugger do Werkzeug executa código; nunca fica exposto na rede
Repositório sem segredos	Todo o código	Não há chaves, senhas ou tokens no código (o push usou o login do próprio Git)

Política CSP aplicada, diretiva por diretiva

Diretiva	Valor	Motivo
default-src	'self'	Padrão restritivo: só o próprio site
script-src	'self' https://cdnjs.cloudflare.com	Scripts do site e o three.js do cdnjs; nenhum script inline
style-src	'self' 'unsafe-inline' https://fonts.googleapis.com	CSS do site e do Google Fonts; <i>unsafe-inline</i> é necessário para os atributos <i>style</i> (ex.: <i>--fill</i> das barras)
font-src	https://fonts.gstatic.com	Arquivos de fonte do Google
img-src	'self' data:	Ícone e máscara de onda embutidos como <i>data:</i> URI
connect-src	'self'	Sem chamadas de rede a terceiros
object-src / base-uri / form-action	'none' / 'self' / 'self'	Bloqueia plugins e evita sequestro de links e formulários

Teste da política: com a CSP real, o navegador não registrou nenhuma violação. Como prova de que o teste enxerga violações, uma cópia com a política propositalmente restritiva foi bloqueada de fato — e o site degradou sem quebrar (o 3D foi desativado com um aviso).

Riscos residuais (transparência)

- **Sem cabeçalhos HTTP no Pages:** o GitHub Pages não permite definir cabeçalhos; por isso a CSP vai em *meta*, e a diretiva *frame-ancestors* (anti-clickjacking) não pode ser usada nesse modo.
- **Dependência de terceiros:** cdnjs e Google Fonts. O SRI protege o script; as fontes revelam o IP do visitante ao Google (hospedar as fontes localmente eliminaria isso).
- **'unsafe-inline' em estilos:** aceito por ser só CSS de apresentação; não permite executar scripts.

12.4 Domínio personalizado: royalnavy.com.br

Resultado da investigação (20/09/2026): o domínio **não está registrado**. Isso foi verificado por vias independentes:

Verificação	Resultado	Interpretação
RDAP do Registro.br (royalnavy.com.br)	HTTP 404	Sem registro no Registro.br
Controle: RDAP de google.com.br	HTTP 200	A consulta funciona e diferencia registrado de livre
DNS (nslookup royalnavy.com.br e www)	Non-existent domain	Não há zona DNS
Variações (royalnavy.br, royal-navy.com.br, royalnavybr.com.br, marinhareal.com.br)	HTTP 404	Também disponíveis na data da consulta

O que não foi feito, e por quê. Registrar um domínio é uma compra em nome de uma pessoa (CPF/CNPJ) com pagamento; só o titular pode fazê-la. Além disso, **o arquivo CNAME não foi publicado:** com o domínio inexistente, o endereço github.io passaria a redirecionar para um endereço que não abre (o site cairia) e, enquanto ninguém o registrasse, qualquer pessoa poderia tomá-lo e receber os visitantes do site.

Trava de segurança criada

O script `enable_custom_domain.py` só grava o CNAME depois de conferir, por DNS, que o domínio já aponta para os quatro IPs do GitHub Pages. Foi testado: recusou `royalnavy.com.br` (não resolve) e `example.com` (resolve para outro servidor), sem gravar nada.

```
python enable_custom_domain.py royalnavy.com.br # ativa (recusa se o DNS ainda não estiver certo)
python enable_custom_domain.py --remove         # volta ao endereço github.io
```

Passo a passo para ativar

- **1. Registrar** royalnavy.com.br em registro.br (conta, CPF/CNPJ, pagamento da anuidade; confirme o valor vigente no site).
- **2. Criar os registros DNS** (tabela abaixo) na zona do domínio.
- **3. Aguardar a propagação** (minutos a algumas horas) e conferir com `nslookup royalnavy.com.br`.
- **4. Ativar no projeto:** rodar o script, depois `git add -A`, `git commit` e `git push`.
- **5. No GitHub** (Settings → Pages): conferir o domínio e marcar **Enforce HTTPS** quando o certificado for emitido (até ~1 h).
- **6. Verificar o domínio** na conta (Settings → Pages → Add a domain, registro TXT), impedindo que terceiros o usem em outro repositório.

Tipo	Nome	Valor	Obrigatório
A	@	185.199.108.153 · 185.199.109.153 · 185.199.110.153 · 185.199.111.153 (um registro por IP)	Sim
AAAA	@	2606:50c0:8000::153 · 2606:50c0:8001::153 · 2606:50c0:8002::153 · 2606:50c0:8003::153	Opcional (IPv6)
CNAME	www	alansouzadev7.github.io	Sim (para o www funcionar)

IPs conferidos na API oficial do GitHub (api.github.com/meta, campo *pages*) em 20/09/2026.

Solução de problemas

Sintoma	Causa provável	O que fazer
O script responde "BLOQUEADO ... não resolve"	Domínio não registrado ou DNS ainda não propagou	Conferir o registro e os registros A; aguardar e repetir
O script responde que resolve para outros IPs	Registros A apontando para outro servidor (ou proxy)	Trocar pelos 4 IPs do GitHub; desligar proxies/CDN que ocultem o IP
Pages mostra "DNS check unsuccessful"	Propagação incompleta ou www sem CNAME	Esperar e usar "Check again"; revisar o CNAME do www
"Enforce HTTPS" desativado	Certificado ainda em emissão	Aguardar até ~1 h; se persistir, remover e recolocar o domínio
O site abre sem estilo/3D depois de ativar o domínio	Cache ou caminhos	Ctrl+F5; os caminhos são relativos e não mudam com o domínio
Quero voltar ao endereço github.io	—	<code>enable_custom_domain.py --remove</code> , commit, push e limpar o domínio em Settings → Pages

12.5 Checklist de publicação

Item	Estado (20/09/2026; domínio reconferido em 26/09/2026)
Código no GitHub (repositório público)	Concluído
Versão estática em docs/ + .nojekyll	Concluído
GitHub Pages ativado (main /docs) e respondendo HTTP 200	Concluído (ativado pelo titular; verificado por curl)
HTTPS	Concluído (automático no endereço github.io)
CSP, SRI e cabeçalhos	Concluído e testado
Modo escuro (v2.0.0): commit, tag e site atualizado e verificado no ar	Concluído (26/09/2026)
Registro de royalnavy.com.br	Pendente — depende do titular (compra)
Registros DNS (A, AAAA, CNAME www)	Pendente — após o registro
enable_custom_domain.py + push	Pendente — após o DNS propagar
Enforce HTTPS e verificação do domínio	Pendente — após ativar o domínio

13. Glossário

Termo	Significado
Canvas 2D	Elemento HTML em que o JavaScript desenha pixels diretamente (usado no hero).
WebGL	API do navegador para desenhar gráficos 3D na GPU. O Three.js é uma biblioteca que a simplifica.
Three.js	Biblioteca JavaScript de 3D: cena, câmera, luzes, geometrias e materiais.
Draw call	Uma ordem de desenho enviada à GPU. Muitas por frame = mais custo de CPU.
InstancedMesh	Objeto Three.js que desenha várias cópias da mesma peça em uma única draw call.
DPR (device pixel ratio)	Razão entre pixels físicos e CSS. Em escala de 125%, DPR = 1,25.
FPS	Quadros por segundo; 60 é o alvo de fluidez em telas comuns.
requestAnimationFrame (rAF)	Agenda um desenho sincronizado com a taxa de atualização da tela.
dt (delta time)	Tempo real entre dois frames; usado para o movimento não depender do monitor.
IntersectionObserver	API que avisa quando um elemento entra ou sai da tela, sem ler layout a cada scroll.
ResizeObserver	API que avisa quando um elemento muda de tamanho.
Compositor	Etapas do navegador que combina camadas na GPU. transform e opacity animam nele sem repintar.
Repaint / Layout	Repintar pixels / recalcular posições. Ambos são caros quando ocorrem em todo frame.
Especificidade CSS	Regra que decide qual declaração vence quando várias se aplicam ao mesmo elemento.
backdrop-filter	Efeito de desfoque do que está atrás de um elemento; caro sobre conteúdo animado.
SRI	Subresource Integrity: hash que o navegador confere para garantir que um script externo não foi alterado.
defer	Atributo do script que o baixa em paralelo e executa após o HTML, sem bloquear a página.
Debugger do Werkzeug	Console interativo do Flask em modo debug; permite executar código no servidor.
prefers-reduced-motion	Preferência do sistema por menos animação; o site a respeita.
inert	Atributo HTML que remove um elemento do foco, cliques e leitores de tela.
GitHub Pages	Hospedagem gratuita de sites estáticos a partir de um repositório do GitHub.
Site estático	Conjunto de arquivos (HTML, CSS, JS) entregues como estão, sem processamento no servidor.
localhost / 127.0.0.1	Endereço da própria máquina. Não é acessível por outras pessoas.
CSP	Content-Security-Policy: regra que diz ao navegador de quais origens a página pode carregar recursos.
HTTPS / TLS	Conexão criptografada entre navegador e servidor; o Pages a fornece automaticamente.
DNS	Sistema que traduz nomes (royalnavy.com.br) em endereços IP.
Registro A / AAAA	Registros DNS que ligam um nome a endereços IPv4 / IPv6.
Registro CNAME	Registro DNS que faz um nome (www) apontar para outro nome (alansouzadev7.github.io).
Propagação de DNS	Tempo para as mudanças de DNS chegarem a todos os servidores (minutos a horas).

Termo	Significado
RDAP	Protocolo de consulta a dados de registro de domínios; usado para checar a disponibilidade.
NXDOMAIN	Resposta DNS "domínio inexistente".
Enforce HTTPS	Opção do Pages que redireciona todo acesso HTTP para HTTPS.
data-theme	Atributo do <html> (light ou dark) que ativa os estilos e cenas do modo escuro.
View Transitions API	API do navegador que fotografa a página antes e depois de uma mudança e permite animar a passagem entre as duas imagens.
clip-path	Propriedade CSS que recorta a área visível de um elemento; aqui, um círculo que cresce a partir do botão.
Web Animations API	element.animate(): anima propriedades por JavaScript, inclusive pseudo-elementos, usando o compositor.
Sprite	Imagem pequena pré-desenhada e reutilizada (brilho, fumaça, silhueta de navio).
Pool de objetos	Conjunto fixo de objetos reaproveitados (partículas, bolhas), para não criar objetos durante a animação.
Mistura aditiva	Modo de desenho em que as cores se somam (globalCompositeOperation = 'lighter'); usado em fogo e brilhos.
Meia resolução	Canvas desenhado com metade da largura e da altura (1/4 dos pixels) e ampliado pelo navegador.
RNPerf	Monitor de tempo de quadro do site que reduz partículas e efeitos do modo escuro em máquinas lentas.
HUD	Elemento fixo sobre a tela (aqui, o medidor de profundidade).
Zonas oceânicas	Camadas do mar por profundidade: epipelágica (até ≈ 200 m), mesopelágica (até ≈ 1.000 m), batipelágica (até ≈ 4.000 m) e abissal; o medidor do site usa esses limites.
Mediana / p95	Valor central de uma série / valor abaixo do qual ficam 95% das medições.
WCAG	Diretrizes de acessibilidade web; o nível AA pede contraste mínimo de 4,5:1 para texto comum.
Tag (git)	Marca permanente em um commit; a v2.0.0 é uma tag.

— Fim do documento —